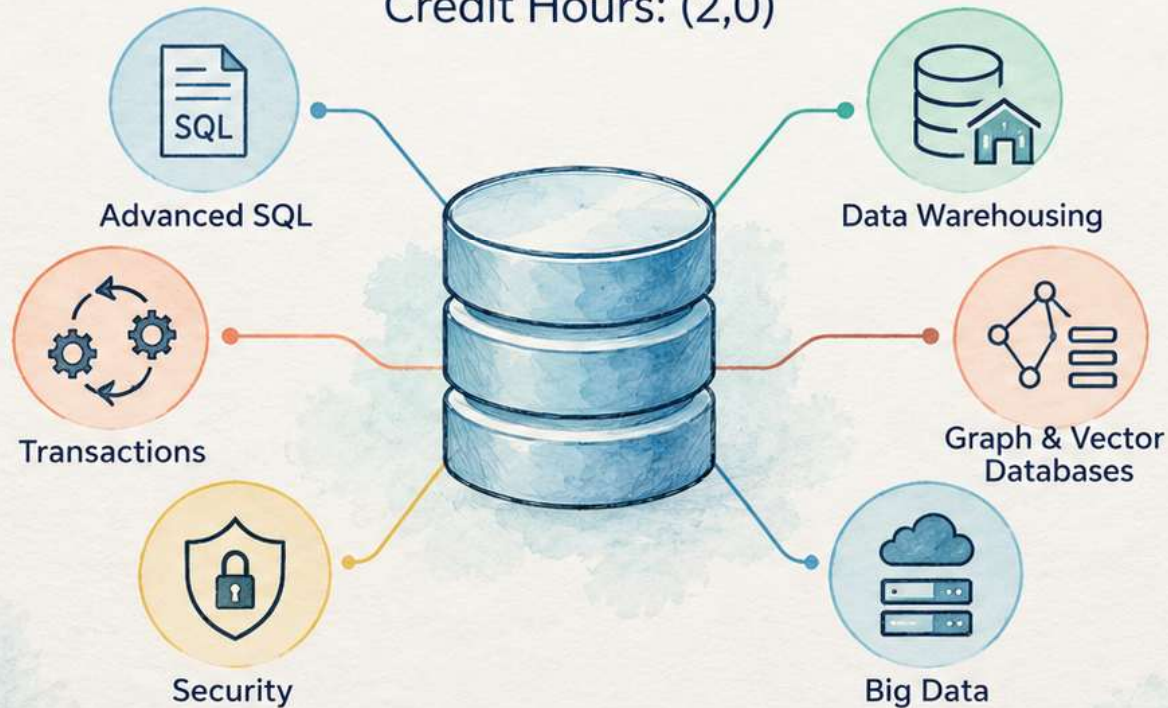


CSC 3108

Advance Database Management Systems

Credit Hours: (2,0)



Course Introduction

DATA
IDEAS
PEOPLE
A BRIGHTER
TOMORROW



Relational Foundations

DATA
RELATIONSHIPS
POWER
REAL-WORLD
APPLICATIONS

A **relational database** organizes data into related tables.

Primary Key



Student

	StudentID	Name	Email	Department
Row 1	S001	Alice Tan	alice@uni.edu	CS
Row 2	S002	Brian Lee	brian@uni.edu	EE
Row 3	S003	Chloe Kim	chloe@uni.edu	CS
Row 4	S004	Daniel Wu	daniel@uni.edu	ME

Columns

Foreign Key
(references Student)

Foreign Key
(references Course)



Enrollment

Student
1 — N
A student can
have many
enrollments.

	EnrollmentID	StudentID	CourseID	Grade
	E001	S001	C101	A
	E002	S001	C102	B+
	E003	S002	C101	A-
	E004	S003	C103	B
	E005	S004	C102	A

Enrollment
N — 1
A course can
have many
enrollments.

Primary Key



Course

	CourseID	Title	Credits	Department
	C101	Database Systems	4	CS
	C102	Distributed Systems	4	CS
	C103	Data Analytics	3	IS
	C104	Operating Systems	4	CS



Tables store entities; **keys** connect records; **constraints** protect validity.

SAME
IDEAS
BRIGHTER
FUTURES



Constraints and Good Table Design

Constraints define valid data; normalization reduces avoidable duplication.

BETTER
DATA
BRIGHTER
SOLUTIONS

A Duplicated Student-Course Table (design with redundancy)

StudentID	StudentName	CourseID	CourseName	Instructor
S001	Alice Tan	CSE101	Database Systems	Dr. Rao
S001	Alice Tan	CSE102	Operating Systems	Dr. Kim
S002	Brian Lee	CSE101	Database Systems	Dr. Rao
S003	Cara Ng	CSE101	Database Systems	Dr. Rao
S003	Cara Ng	CSE103	Computer Networks	Dr. Patel

! Update anomaly

If the course name changes from "Database Systems" to "Data Management Systems", we must update every row for CSE101. If we miss a row, the data becomes inconsistent.

Same fact
repeated in
multiple rows

Normalized Design (3 related tables)

STUDENT

<u>StudentID</u>	StudentName	Email	Major
S001	Alice Tan	alice@uni.edu	CS
S002	Brian Lee	brian@uni.edu	CS
S003	Cara Ng	cara@uni.edu	IS

PRIMARY KEY

Uniquely identifies each student.

COURSE

<u>CourseID</u>	CourseName	Instructor	Credits
CSE101	Database Systems	Dr. Rao	3
CSE102	Operating Systems	Dr. Kim	3
CSE103	Computer Networks	Dr. Patel	4

UNIQUE

Course names must be unique.

NOT NULL

Every course must have an instructor.

CHECK

Credits must be between 1 and 6.

ENROLLMENT

<u>StudentID</u>	<u>CourseID</u>	EnrolmentDate	Grade
S001	CSE101	2024-08-26	A
S001	CSE102	2024-08-27	B+
S002	CSE101	2024-08-26	A-
S003	CSE101	2024-08-28	B
S003	CSE103	2024-08-28	A

StudentID
(FOREIGN KEY)
references
STUDENT(StudentID)

CourseID
(FOREIGN KEY)
references
COURSE(CourseID)



Revision Notes

1



Choose keys

Identify primary keys and candidate keys.

2



Enforce valid relationships

Use foreign keys and constraints (UNIQUE, NOT NULL, CHECK).

3



Store each fact in its appropriate place

Normalize to reduce duplication and avoid anomalies.

Relational Thinking: Quick Revision

SQL retrieves and combines data from related tables.



Relational data turns separate facts into meaningful answers.

SAME DATA.
DIFFERENT QUESTIONS.
THAT'S SQL.

Query-writing guide, not execution order.

1 SELECT

Choose the columns (what to show).

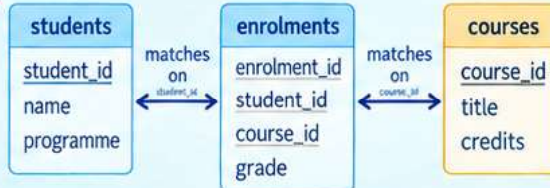
e.g. student name and course title

student_name	course_title
Aisha Khan	Databases
Ben Lee	Operating Systems
Chloe Tan	Machine Learning
...	...

2 FROM / JOIN

Specify the tables and how they are related.

e.g. students joined with enrolments and courses



3 WHERE

Filter the rows (which data to keep).

e.g. only Computer Science students taken in 2024



Keeps only rows that match the condition.

4 GROUP BY

Summarize the results (group rows).

e.g. count students per course

course_title	num_students
Databases	120
Operating Systems	85
Machine Learning	64
...	...



A closer look: JOIN (combines related rows)

students	
student_id	name
S001	Aisha Khan
S002	Ben Lee
S003	Chloe Tan
...	...

Matching key
(student_id)

Links related rows
from both tables.

enrolments	
student_id	course_id
S001	C101
S002	C102
S003	C103
...	...



A closer look: Subquery (use a query within another query)

Using a subquery

Use a query within another query to get a value or a set of values that the main query uses.

Often expresses the same result.

Same idea, different structure

A subquery runs inside another query and provides data that the main query can use.



Key revision points:

JOIN: combine related rows; **Subquery:** use a query within another query.



Different clauses.
A more complete answer.

Advanced SQL Queries

Advanced querying expresses multi-step questions clearly in SQL.

CTE = named query step

A common table expression (CTE) defines a temporary result used in a larger query.



Window function = calculation across related rows

A window function computes a value using a group of related rows without collapsing them.

```
SELECT e.name,
       e.dept,
       e.salary,
       RANK() OVER (PARTITION BY e.dept
                   ORDER BY e.salary DESC) AS rnk
FROM employees e;
```

name	dept	salary	rnk
Ava	Sales	90000	1
Ben	Sales	75000	2
Cara	Sales	60000	3
Dan	HR	80000	1

View = saved query definition

A view stores a query and can be queried like a table.

```
CREATE VIEW high_earners AS
SELECT e.id, e.name, e.salary
FROM employees e
WHERE e.salary > 75000;

SELECT *
FROM high_earners;
```



Use these tools for readable queries, rankings, and reusable reports.

Window Functions Preserve Rows



Window functions calculate across a set of rows without collapsing each row.

Input: Student Marks		
student	course	mark
Ana	DB	80
Ben	DB	70
Cara	AI	90
Dan	AI	60

GROUP BY
One average
per course

Average mark per course

course	avg_mark
DB	75
AI	75

Window Function
Each student row
retained

Each row with course average

student	course	mark	avg_mark
Ana	DB	80	75
Ben	DB	70	75
Cara	AI	90	75
Dan	AI	60	75



PARTITION BY defines groups; **ORDER BY** defines sequence; rows remain visible.



Choose the Right Query Tool

Choose a query structure according to the result you need.

Summarize groups

GROUP BY

Combine rows and compute aggregates for each group.

Programme	Avg Mark
CS	75
DS	68

Keep rows plus calculations

Window function

Add calculations without collapsing rows.

Student	Programme	Mark	Programme Avg
Ava	CS	80	75
Ben	CS	70	75
Chao	DS	68	68

What do you want to do?

Name intermediate step

CTE

Define a temporary result for clarity.

```
WITH prog_avg AS (  
  SELECT programme, AVG(mark) AS avg_mark  
  FROM enrolments  
  GROUP BY programme  
)  
SELECT * FROM prog_avg;
```

Reuse query definition

View

Save a query for repeated use.

Saved SELECT definition

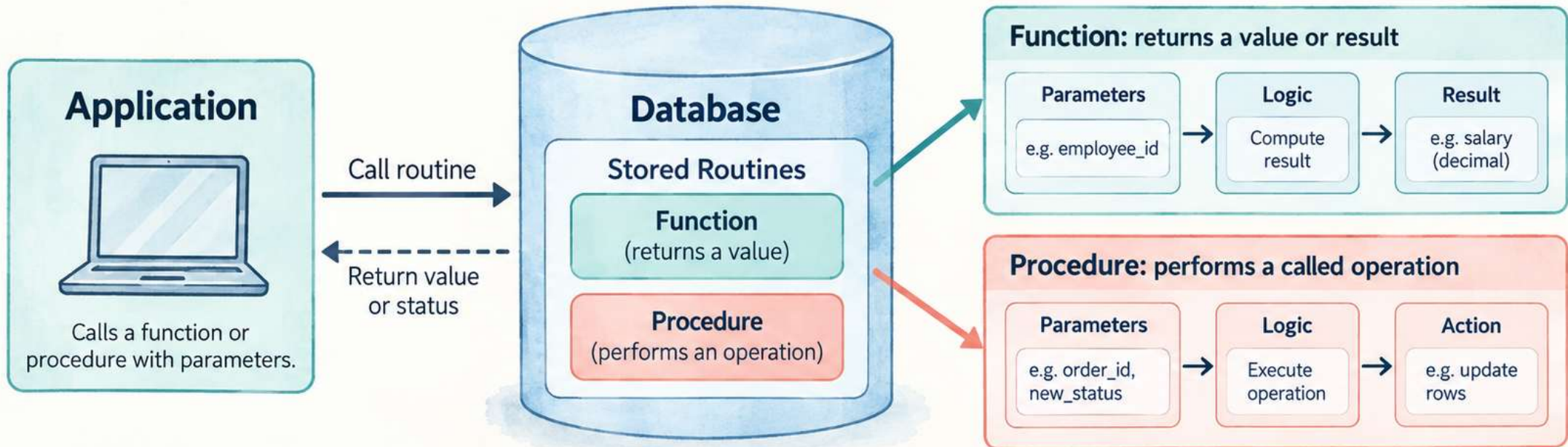


Practice: rank students within programmes and compare marks with course averages.



Database Programming

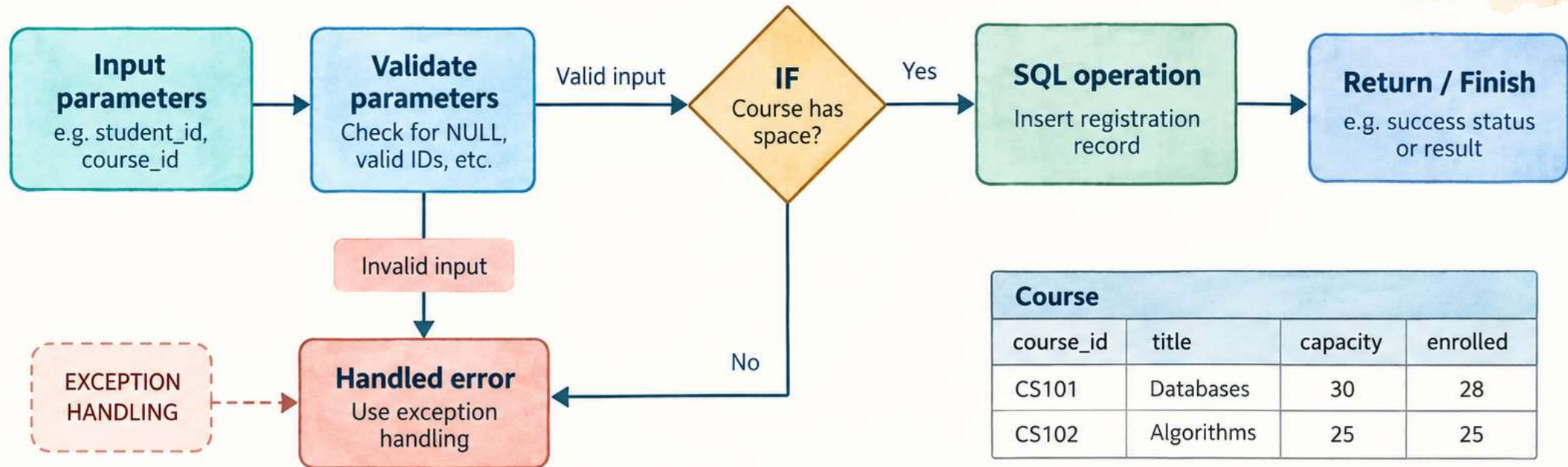
Stored routines execute named logic inside the database.



Revision: Keep routines small; define inputs; choose database versus application logic deliberately.

Inside a Stored Routine

A routine combines SQL with control flow.



Example: register a student only when a course has space.

Functions and Procedures: Revision

Functions and procedures package reusable database operations.



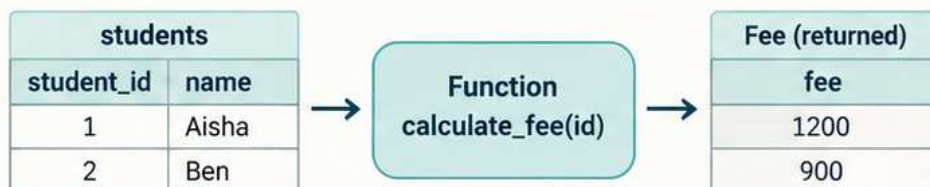
DATA
MODELS
DRIVE
POSSIBILITIES

Function

Returns a value

- Used in expressions or queries (where supported).
- Typically for calculations or lookups.

```
SELECT s.student_id,
       calculate_fee(s.student_id) AS fee
FROM students AS s;
```



Used inside a query to compute the fee.

Procedure

Performs an action

- Invoked with CALL in PostgreSQL.
- Used for operations that change data (no return value).

```
CALL register_student(1, 'CS101');
```



Revision note:

Check parameter types, successful outcomes, and failure cases.



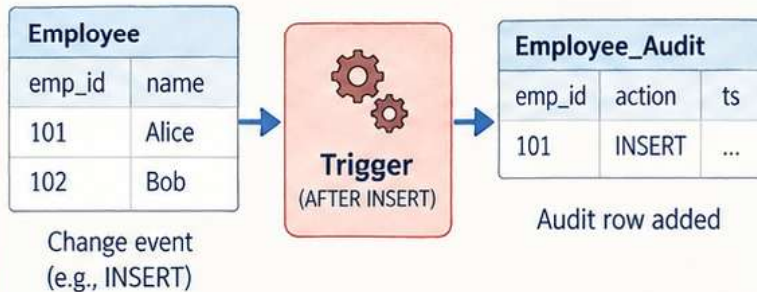
Triggers, Cursors, and Dynamic SQL

Week 4 • Slide 1 • Advanced DBMS
10/45

These tools automate reactions, process rows, or construct queries at runtime.

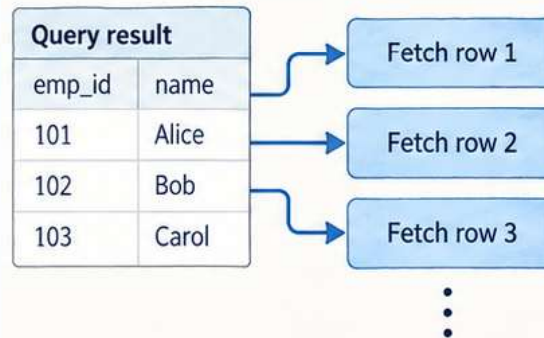
Triggers

React to data changes automatically.



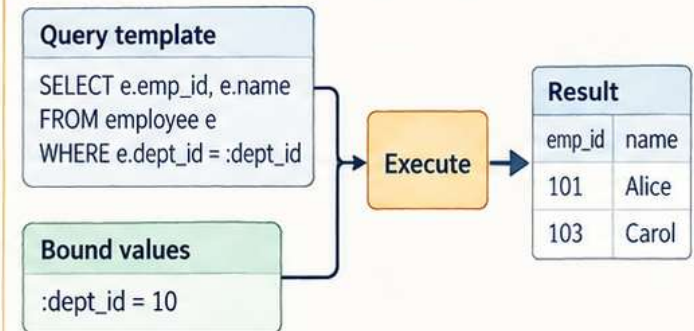
Cursors

Process result sets row by row.



Dynamic SQL

Construct and execute queries at runtime.



Revision: Prefer simple SQL when it solves the problem.



Choose Constraints Before Triggers

Advanced
DBMS

Constraints express data rules; triggers respond to database events.

Built-in validity rule → Constraint

Example: marks must be between 0 and 100.

Student	
student_id	mark
S101	60
S102	85
...	...



CHECK constraint
mark $\geq$ 0
AND mark $\leq$ 100



Invalid values are rejected.

Record who changed a mark → Audit trigger

Example: keep an audit trail when a mark is updated.

OLD (before)	
student_id	mark
S101	60

UPDATE

NEW (after)	
student_id	mark
S101	70



Audit record			
student_id	old_mark	new_mark	changed_by
S101	60	70	U123



Trigger inserts audit record on update event.



Revision: Use CHECK for ranges; FOREIGN KEY for references; triggers for justified event logic.



Set-Based SQL and Safe Dynamic Queries

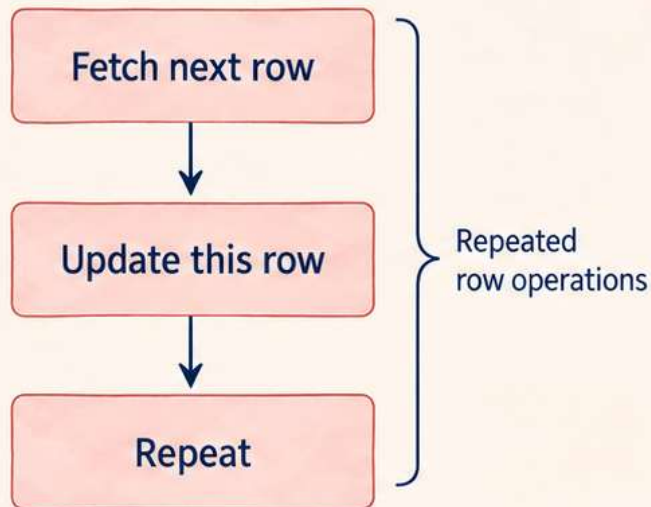
Set-based SQL handles sets; bound parameters separate values from SQL code.



DATA
QUERIES
BETTER SYSTEMS

Loop over rows (row-by-row)

Fetch rows and update each one.



One set-based UPDATE

Update all matching rows in a single statement.

```
UPDATE employees e
SET salary = salary * 1.10
WHERE e.dept_id = 10;
```



Database

One UPDATE
affects all
matching rows

Employees (before and after)

id	name	dept_id	salary (before)	salary (after)
1	Ann	10	50,000	55,000
2	Ben	10	60,000	66,000

Safe Dynamic Query Pattern

Use bound parameters for values.
Validate dynamic identifiers with an allowlist.

Query template + bound value

```
SELECT e.id, e.name, e.salary
FROM employees e
WHERE e.dept_id = :dept_id;
(client-library syntax)
```

Bound value

:dept_id = 10

Identifier allowlist (example)

Only allow expected column names.

Allowed columns:

- id
- name
- salary
- dept_id

Reject others:

- Any unapproved identifier



Revision: Avoid unnecessary cursors; bind values; validate dynamic identifiers.

Indexes and Query Performance

13/45

An index is an extra structure that helps locate qualifying data.

Book index (analogy)



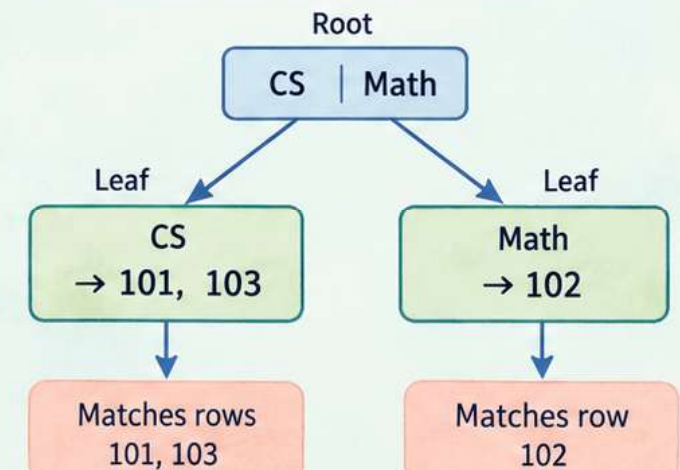
Index	
Apple	12
Database	45
Index	87
Query	121
Transaction	190

Look up a term to find the relevant pages quickly.

Database table

Student		
id	name	dept
101	Ana	CS
102	Ben	Math
103	Cara	CS
104	Dan	Physics

Index (B-tree on dept)

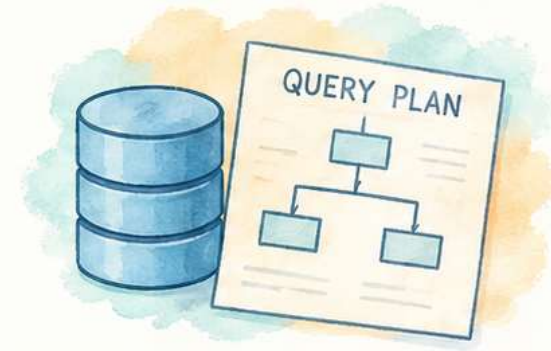


Indexes can speed reads but cost storage and maintenance during writes.



Reading a Simple Query Plan

A **query plan** describes how the database will execute a query.



Sequential Scan (scan all rows)

Employees

id	name	dept_id
1	Ana	10
2	Ben	20
3	Cara	10
...	...	...

Seq Scan

Employees
(est. rows: 1,000)

Actual time:
0.85 ms

Filter

dept_id = 10
(est. rows: 200)

Actual time:
0.15 ms

Index Lookup (find matching rows)

Employees

id	name	dept_id
1	Ana	10
2	Ben	20
3	Cara	10
...	...	...

Index Scan

idx_employees_dept
(est. rows: 200)

Actual time:
0.12 ms

Fetch Matching Rows

(est. rows: 200)

Actual time:
0.30 ms



A **sequential scan** can be efficient for small tables or queries returning many rows.

Indexing: Quick Revision

Choose indexes for real query patterns and verify the effect.



1. Identify slow query

Use logs or monitoring to find a query with high latency.



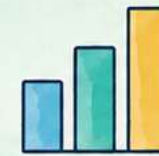
2. Inspect plan

Check how the query is executed (e.g. scans vs index seeks).



3. Add suitable index

Create an index that matches the query predicates and order.



4. Measure

Run the query again and compare execution time and plan.



5. Retain or revise

Keep the index if it helps; otherwise adjust or try a different design.

Composite index example

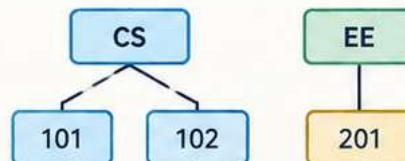
Create an index on (programme, student_id) to support queries filtering by programme and then student_id.

Students (sample rows)

student_id	programme	name
101	CS	Ali
102	CS	Bo
201	EE	Chen

Index on
(programme, student_id)

Index (programme, student_id)



Revision note

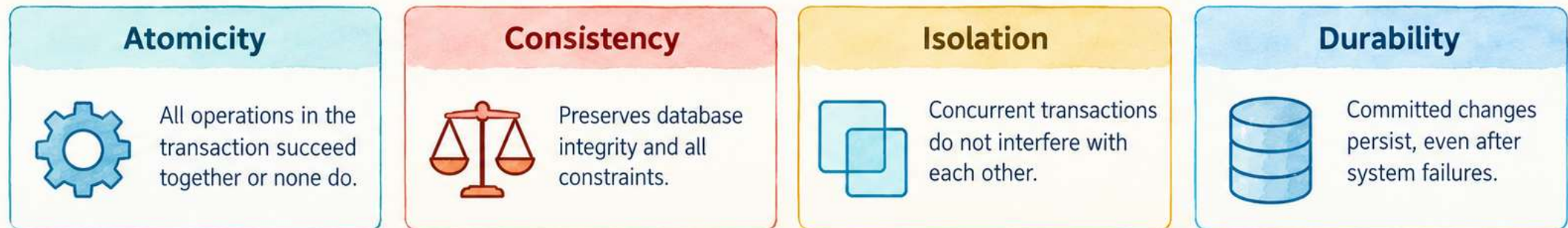
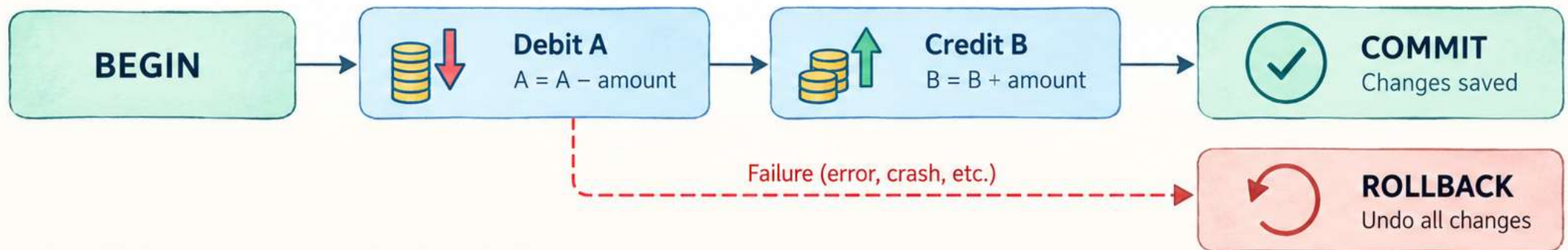


More indexes are not always better; measure both read benefit and write cost.

Transactions and ACID



A transaction groups operations into one logical unit of work.



Revision note: A transfer must not leave only the debit applied.

Schedules and Serializability

A **schedule** orders operations from multiple transactions.



Serial Schedule (T1 then T2)

Step	T1	T2
1	R(A)	
2	W(A)	
3	R(B)	
4	W(B)	
5		R(A)
6		W(A)

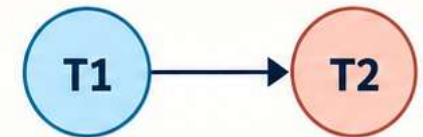
All operations of T1 complete before T2 starts.

Interleaved Schedule

Step	T1	T2
1	R(A)	
2	W(A)	
3		R(A)
4	R(B)	
5		W(A)
6	W(B)	

Operations from T1 and T2 are interleaved.

Conflict Graph



Acyclic: conflict-serializable.

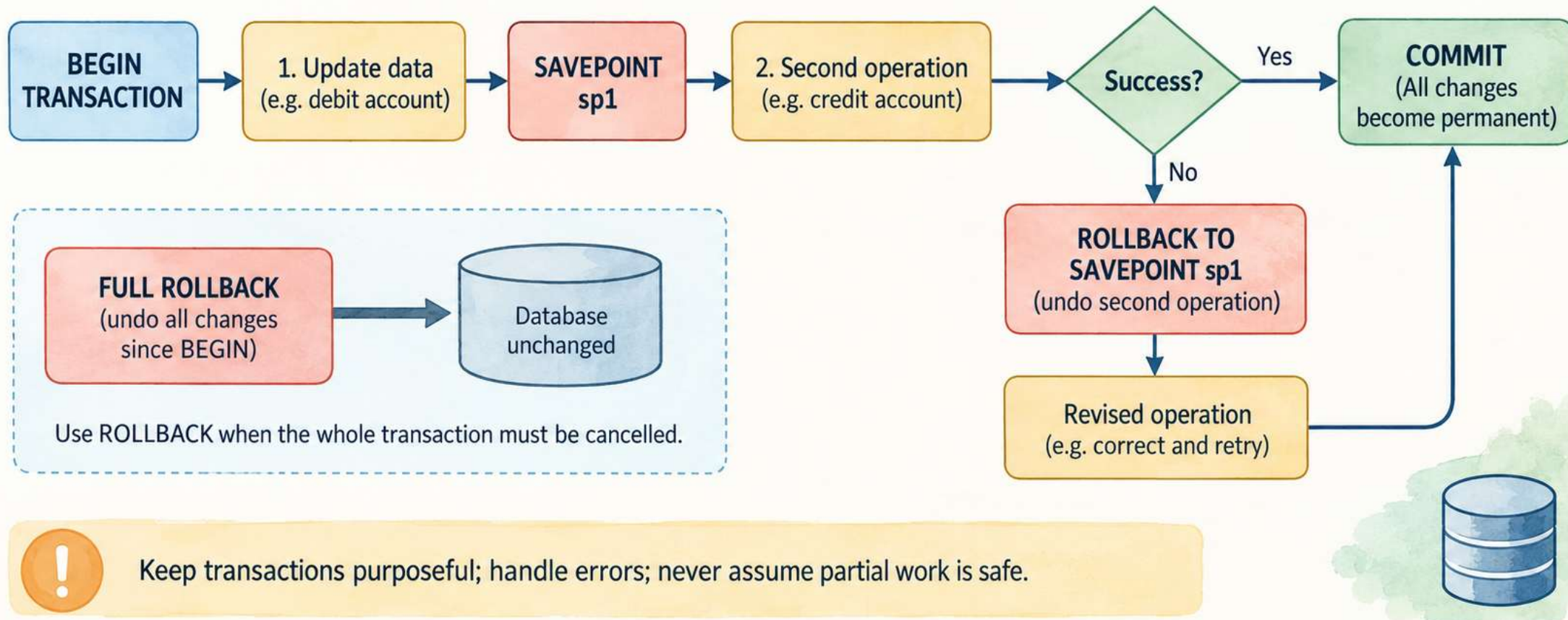
Legend

R = read
W = write



Transaction Boundaries: Revision

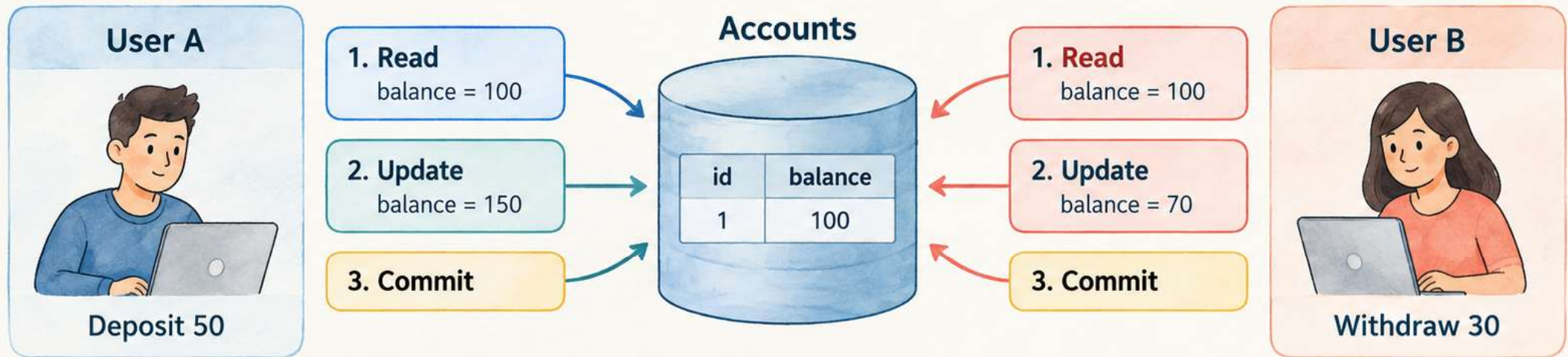
Commit makes a successful transaction permanent; rollback cancels its uncommitted changes.



Keep transactions purposeful; handle errors; never assume partial work is safe.

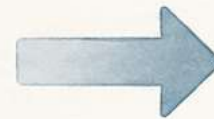
Concurrency in Multi-User Databases

Concurrency lets transactions overlap while controlling interference.



Lost-update scenario

Both users read 100. A commits 150, then B commits 70. A's update is lost; final balance is 70.



Final state

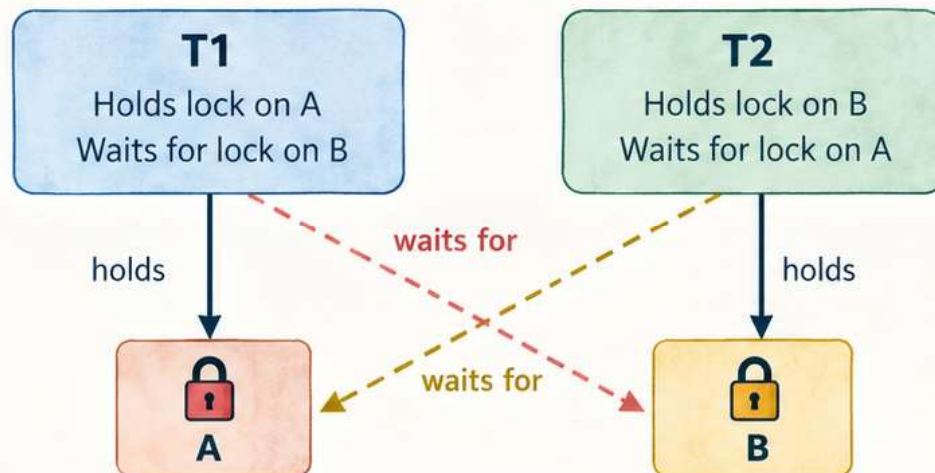
id	balance
1	70

Correctness depends on isolation, locking, and application transaction design.

Locks, Deadlocks, and MVCC

Locks coordinate access; MVCC lets reads use visible row versions.

Deadlock with Locks



Circular wait → deadlock

MVCC and Snapshot Reads

Row versions (same row)

Newer row version

id	value	xmin
1	'B'	105

not visible

Old committed version

id	value	xmin
1	'A'	90

sees

Transaction T3
Snapshot at 100
Reads visible version



Revision note: Keep lock order consistent; keep transactions short; retry aborted transactions when appropriate.

Isolation: Quick Revision

Isolation levels trade permitted interference against coordination and retry costs.

Read Uncommitted



- May see uncommitted changes (dirty reads)
- Lowest coordination

Read Committed



- Sees only committed data
- No dirty reads

Repeatable Read



- Same rows read consistently within a transaction
- Prevents non-repeatable reads

Serializable



- Equivalent to some serial execution
- Prevents phantoms
- Highest coordination



PostgreSQL treats Read Uncommitted as **Read Committed**.

Weaker isolation

Stronger isolation

Snapshot vs. Serial Outcome

Snapshot view (both transactions)

Account

id	balance
1	100
2	100

Both transactions read the same snapshot (e.g. balance = 100 for each account).



One possible serial outcome

Account

id	balance
1	90
2	110

Equivalent to some serial order (e.g. T1 then T2).



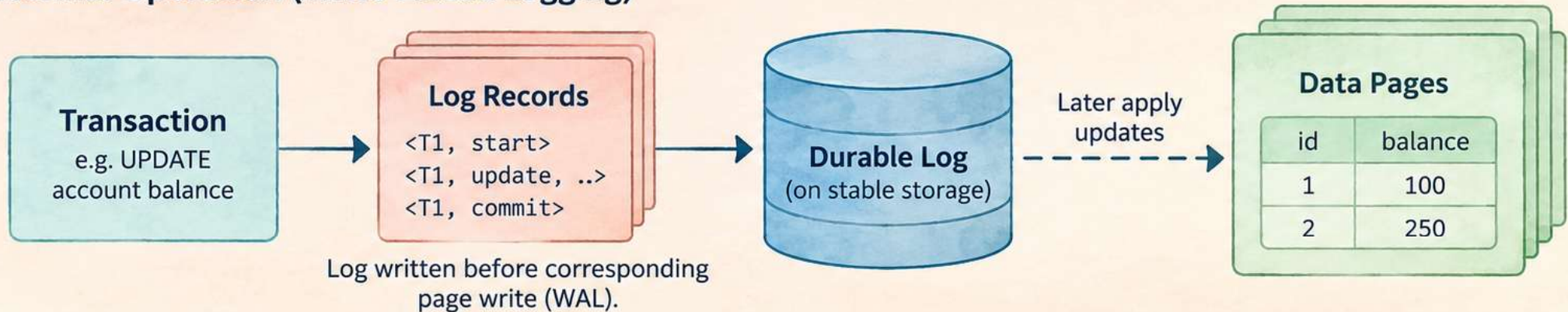
Revision note

Serializable transactions may need retries; stronger isolation is not simply faster or slower in every workload.

Recovery and Backups

Recovery restores a valid database state after failure; backups preserve recoverable copies.

Normal Operation (Write-Ahead Logging)



Backup and Restore

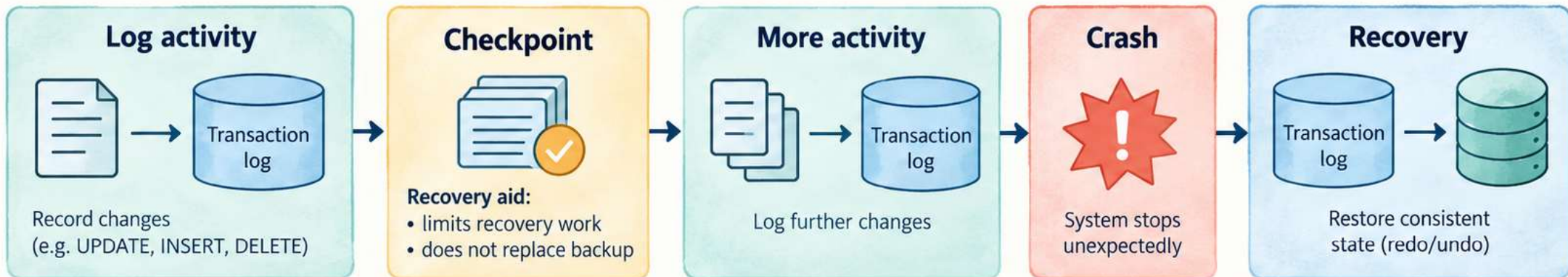


Key Point

Logs and backups serve related but different purposes.

Logs and Checkpoints

Write-ahead logging records changes before affected data pages are written.



Recovery strategies (conceptual)

Redo

- Reapply committed changes from the log
- Engine-dependent implementation



Undo

- Roll back uncommitted changes using log records
- Engine-dependent implementation



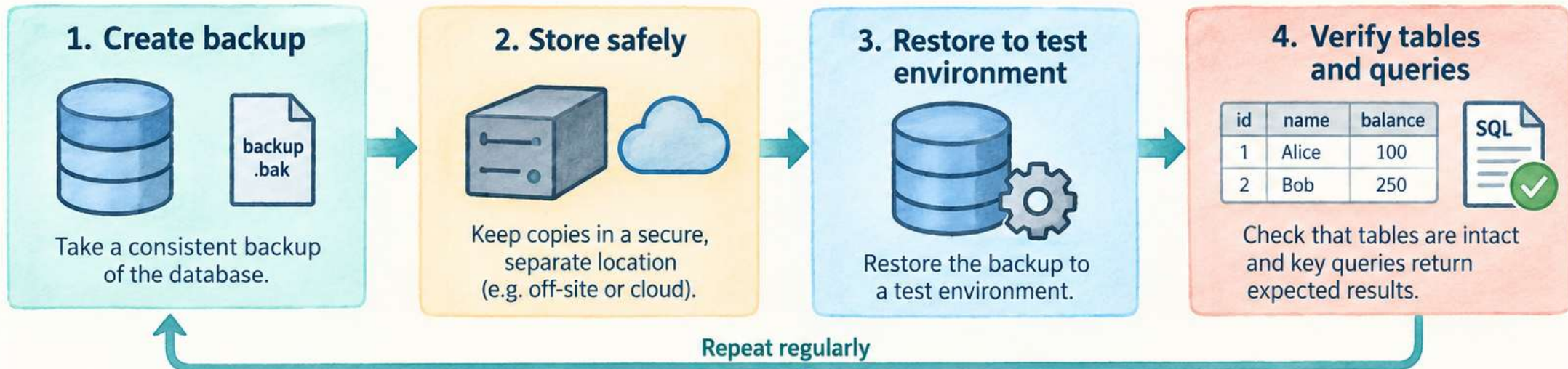
Revision note



Deferred and immediate update are conceptual recovery strategies; implementation differs by DBMS.

Recovery Readiness: Revision

A backup is useful only when it can be restored correctly.



Crash recovery vs Restore from backup

	Crash recovery	Restore from backup
Purpose	Recover from a recent failure using logs.	Rebuild the database from a backup copy.
Data loss	Usually minimal (to last commit).	Up to last backup.
Downtime	Typically shorter.	Usually longer.

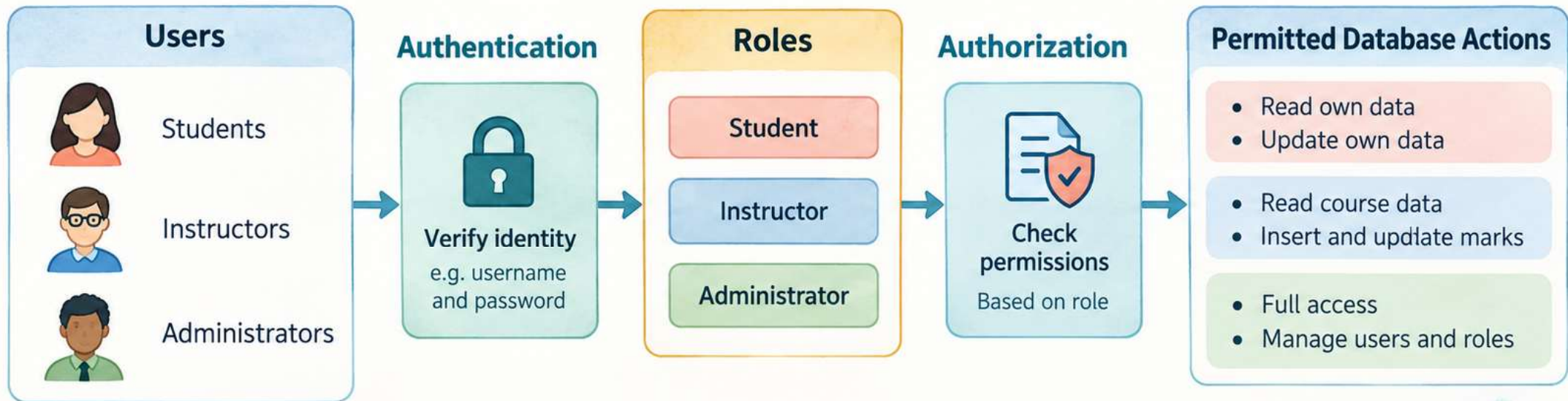
Revision



Define acceptable data loss and downtime; practise a small restore.

Database Security

Database security controls who can access data and what they can do.



Revision:

Grant only the permissions needed for a task.



Preventing SQL Injection

Parameterized queries keep user values separate from SQL structure.

Unsafe: string concatenation

```
sql = "SELECT * FROM users "  
    + "WHERE username = '"  
    + input + "'"
```



User input becomes part of the SQL command.

Safe: parameterized query



User input value
"alice"

Bound value



SQL template

```
SELECT * FROM users  
WHERE username = ?
```

Placeholder syntax depends on client library.

SQL structure



Parameterized execution



Database

Matching result

id	username	role
1	alice	student

Query runs with bound value

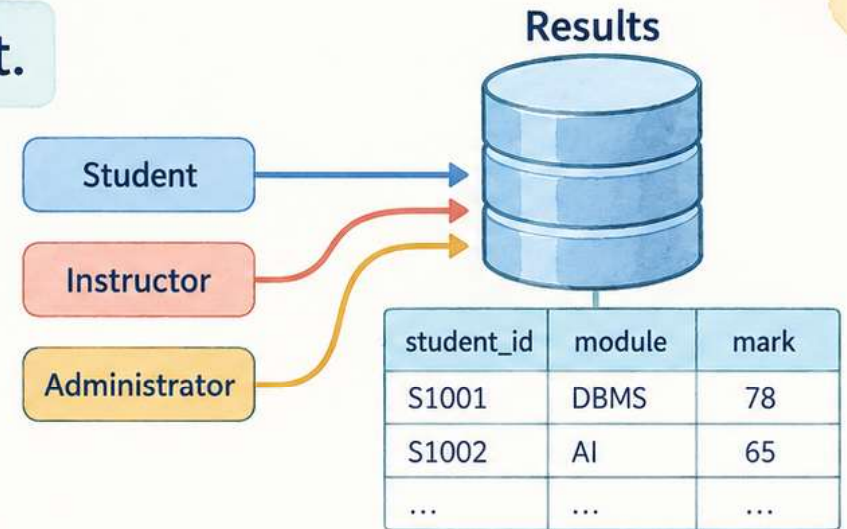


Bind data values; allowlist dynamic identifiers; avoid unnecessary privileges.

Security: Quick Revision

Use layered controls and review who can do what.

Role	View own results	Manage assigned marks	Manage access
 Student	✓	✗	✗
 Instructor	✗	✓	✗
 Administrator	✗	✗	✓



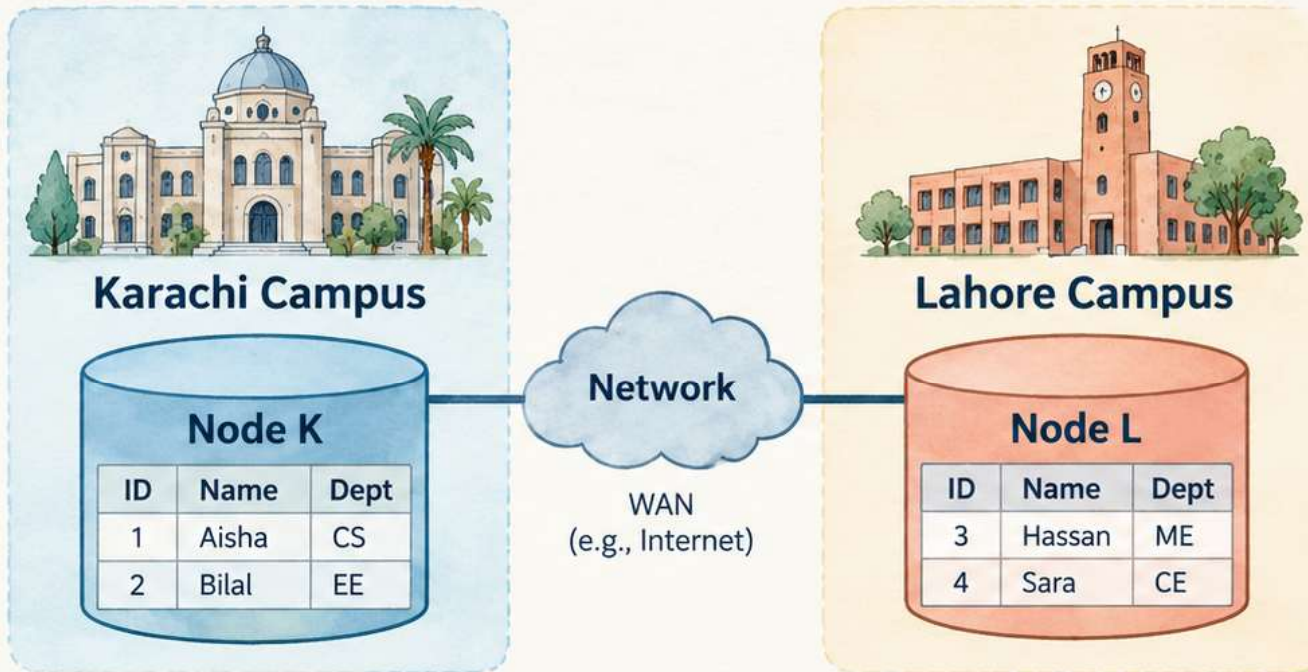
Row-specific access (e.g. a student only sees their own results) needs additional enforcement beyond table grants.



Revision: Least privilege; parameterized queries; audit meaningful changes; review access.

Distributed Database Fundamentals

A distributed database manages data across multiple locations or nodes.



Revision: Distribution introduces network delays, failures, and coordination choices.

Partitioning (Different rows)

Data is split across nodes; each node stores a subset of rows.

Node K	
ID	Name
1	Aisha
2	Bilal

Node L	
ID	Name
3	Hassan
4	Sara

Replication (Copies of the same data)

Each node stores a full copy of the data.

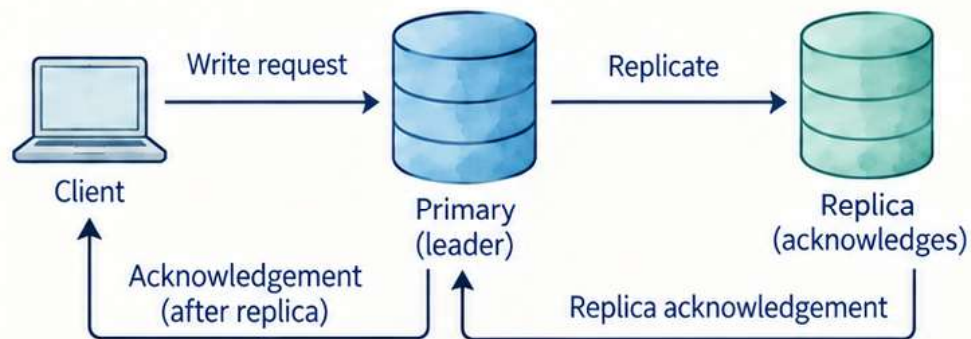
Node K	
ID	Name
1	Aisha
2	Bilal

Node L	
ID	Name
1	Aisha
2	Bilal

Replication and Consistency

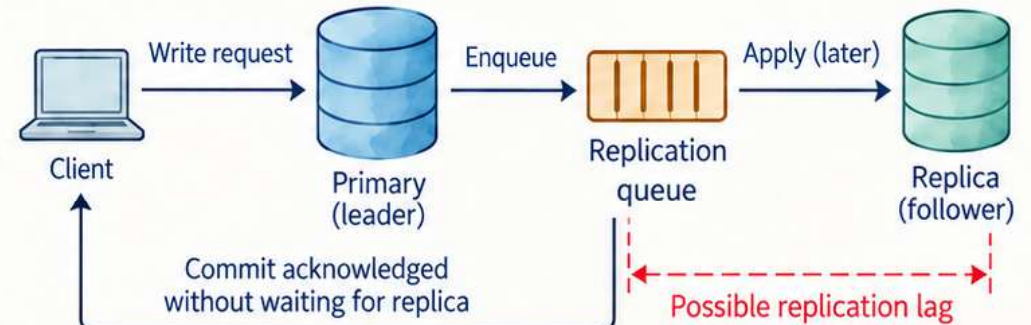
Replication keeps copies of data; consistency describes what readers can observe.

Synchronous (acknowledge after replica)



Waits for the required replica acknowledgement; receive, flush, or apply depends on configuration.

Asynchronous (queue then replica)



Replica may be behind the primary.



Synchronous and asynchronous designs have different latency and failure trade-offs.



Distributed Design: Revision

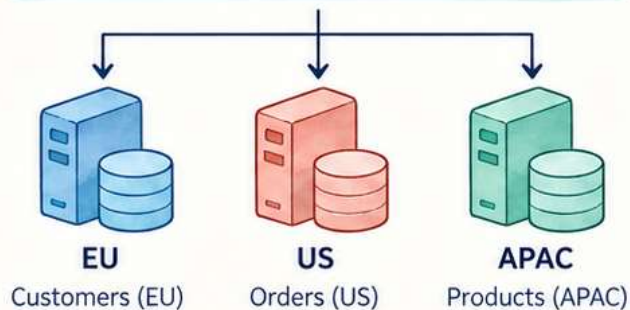
Understand trade-offs;
defer distributed optimizer
internals and cluster administration.

Choose placement and replication based on application requirements.

Decision Canvas

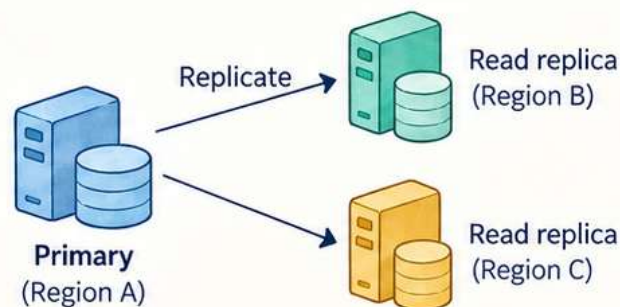
1. Campus ownership → Partitioning

Where does the data naturally belong? Partition to keep related data close.



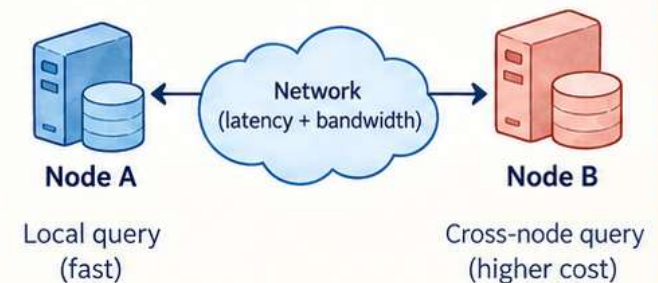
2. Availability / read copies → Replication

Need higher availability or more read throughput? Keep replicated copies.



3. Cross-node queries → Network cost

Queries that access multiple nodes incur network cost. Minimise cross-node data movement.



Large-scale ecosystem examples:



Hadoop

Distributed storage
and processing

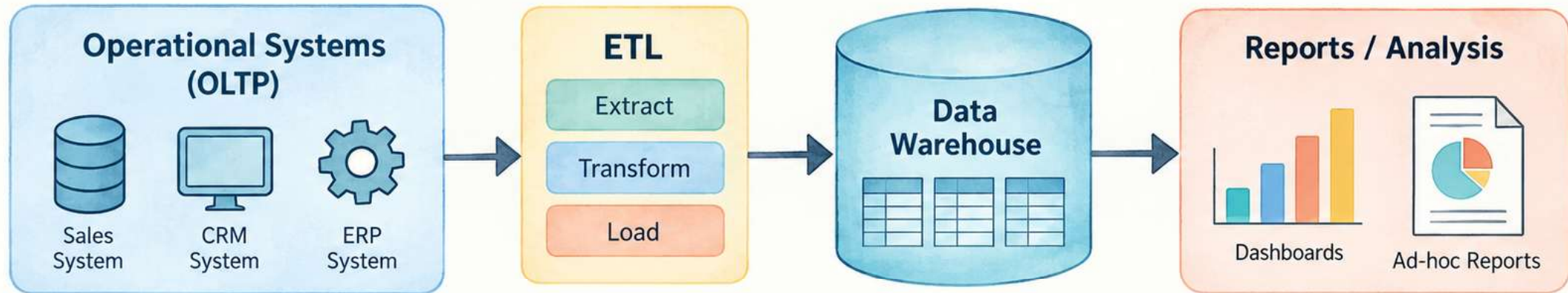


Spark

Distributed data
processing

Data Warehousing

A data warehouse organizes integrated historical data for analysis.



OLTP vs OLAP

OLTP – Daily Transactions

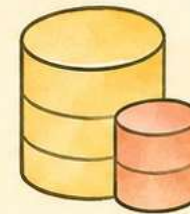
OrderID	CustomerID	Amount
1001	C001	120.00
1002	C002	75.50

- Supports day-to-day operations
- Many short transactions
- Frequently updated data

OLAP – Analytical Questions

Year	Product	Total Sales
2023	Laptops	1,250,000
2023	Phones	980,000

- Supports complex analysis
- Reads large integrated data
- Optimized for aggregation and trends



Data Mart

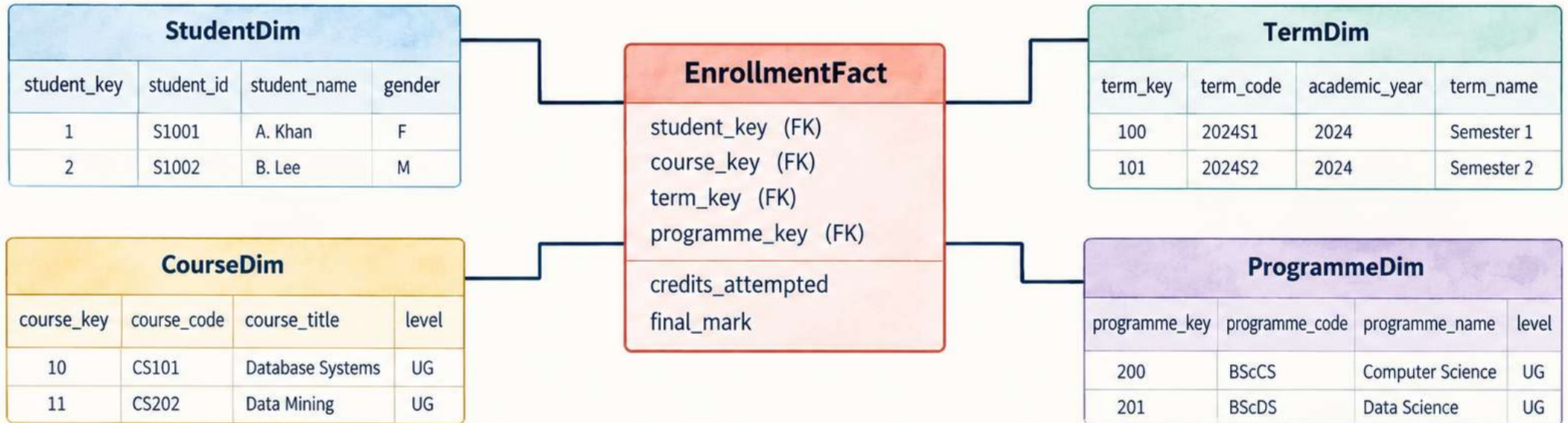
A data mart focuses on a business subject or department.



Star Schema and Grain



The **grain** states exactly what one fact-table row represents.



Grain (explicit)

One student enrollment in one course offering (a specific student, course, and term).



Measures

- credits_attempted
- final_mark

Averages need appropriate interpretation.



Revision note

Define grain first; separate descriptive dimensions from measurable facts.



Warehouse Modelling: Revision

Dimensional models make analytical questions easier to express.

33 / 45

Week 11

Slide 3

1. Question

e.g. Total sales by product category and month?

2. Grain

One row per sale (order line).

3. Dimensions

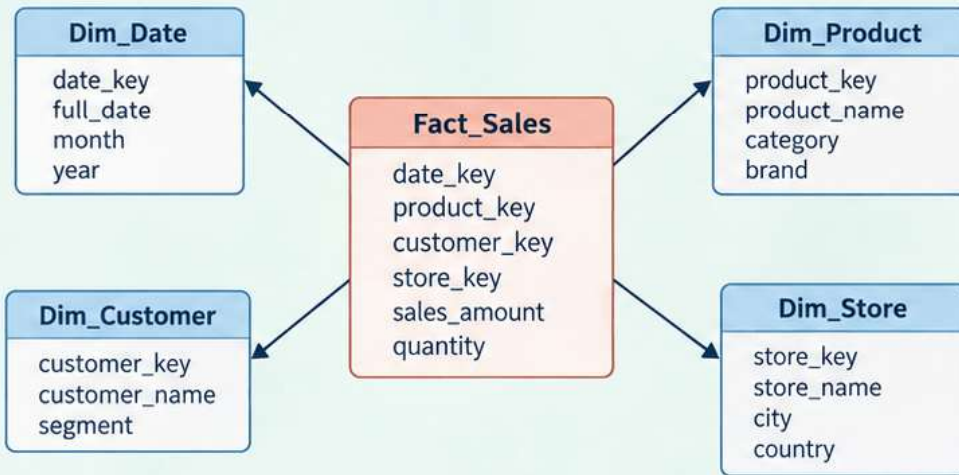
Date, Product, Customer, Store.

4. Measures

Sales amount, Quantity.

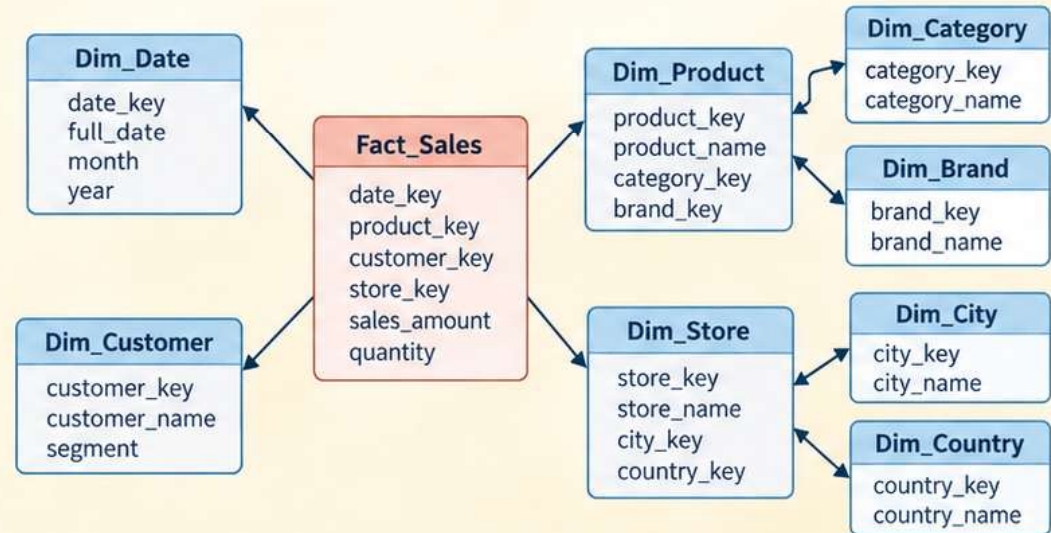
Star Schema

Direct dimensions (denormalised)



Snowflake Schema

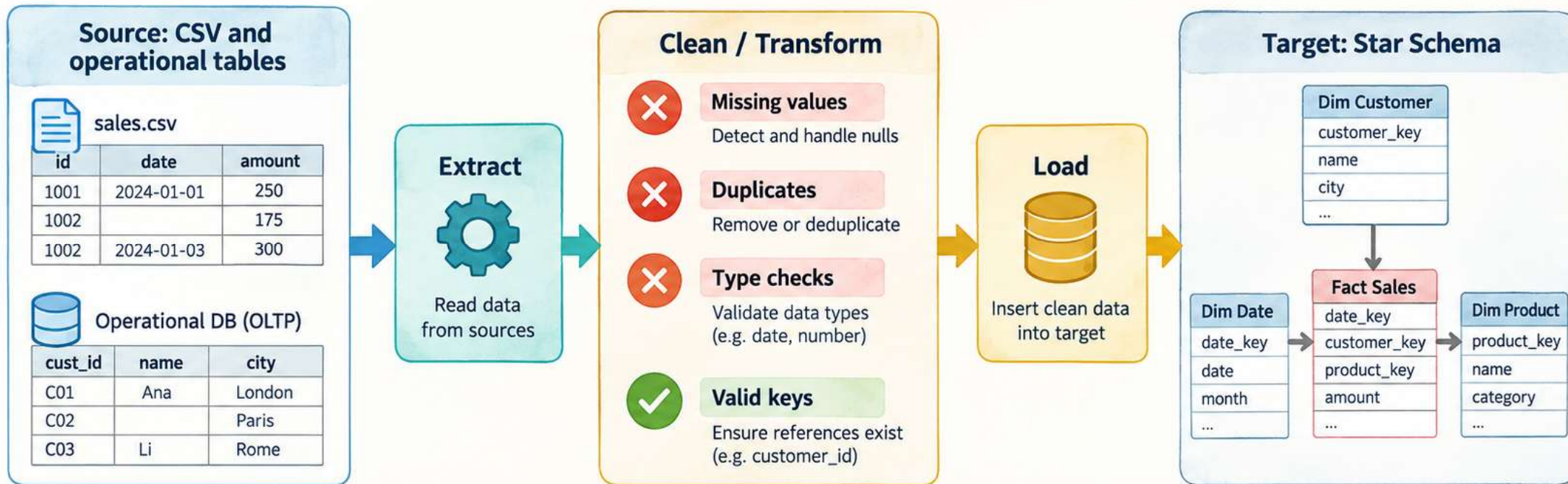
Normalised dimension branches



Revision: Avoid mixing grains; choose meaningful aggregations; start with one small star.

ETL and Data Quality

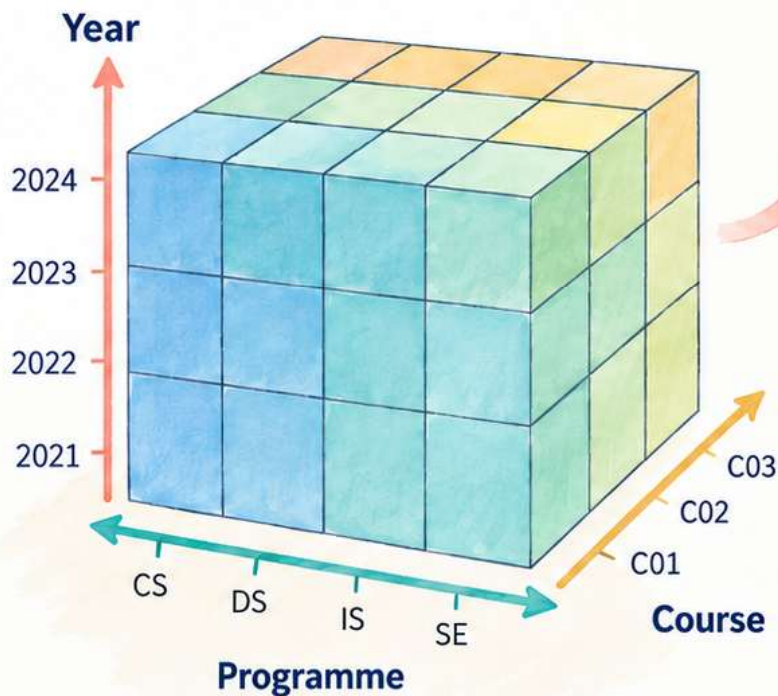
ETL extracts data, transforms it into a consistent form, and loads a target.



Revision: Record rejected rows and reconcile source and loaded counts.

Analytical Queries and OLAP

Analytical queries summarize data across dimensions.



Roll-up: term totals to year

Year	Total Enrolments
2023	1,240
2024	1,310

Aggregate terms
(e.g. Spring, Autumn)
to the year level.

Drill-down: programme totals to course detail

Programme	Course	Enrolments
CS	C01	120
CS	C02	95
CS	C03	80

From programme
totals to individual
course detail.



Use **GROUP BY**
and aggregates;
select the correct
level of detail.



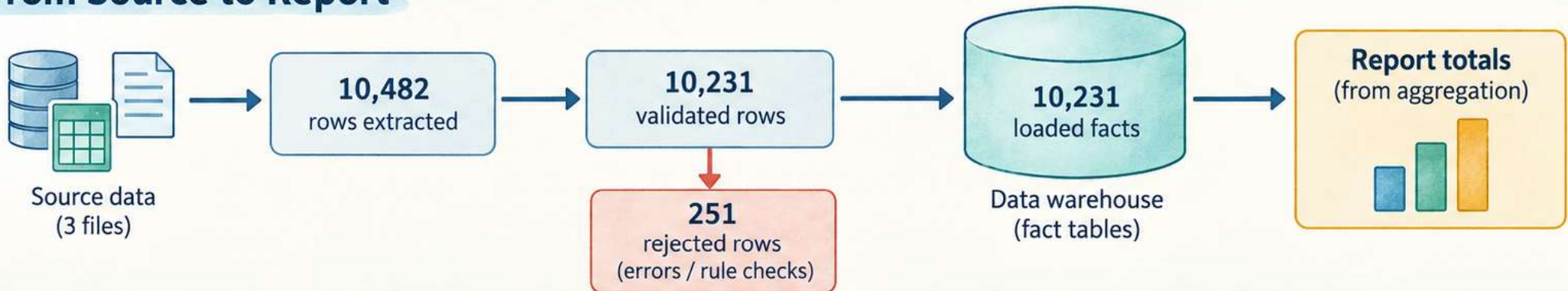
Not every measure
is additive across
dimensions (e.g. average
marks). Choose
appropriate aggregates.

ETL and Analytics: Revision

A trustworthy report needs both correct data and correct aggregation.



From Source to Report



Key Analytical Questions

- 1 What is the enrollment by term? 
- 2 What are the credits by programme? 
- 3 What are the average marks by course? 

Revision Checklist

- ✓ Verify source and target counts.
- ✓ Check key integrity and joins.
- ✓ Confirm data grain matches the analysis.
- ✓ Validate aggregation logic and totals.

Revision note: Verify counts, keys, grain, and aggregation before interpreting results.



NoSQL and Graph Databases

NoSQL includes several models suited to different access patterns.



Key-Value Store

Simple key-based lookup for fast access.

Key	Value
user:101	Alice
user:102	Bob
user:103	{...}



Document Store

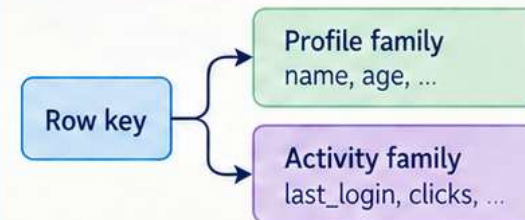
Stores rich, nested records (e.g. JSON documents).

```
{
  "id": 101,
  "name": "Alice",
  "address": {
    "city": "London",
    "zip": "E1 6AN"
  },
  "skills": ["DBMS", "AI"]
}
```

Column-Family Store

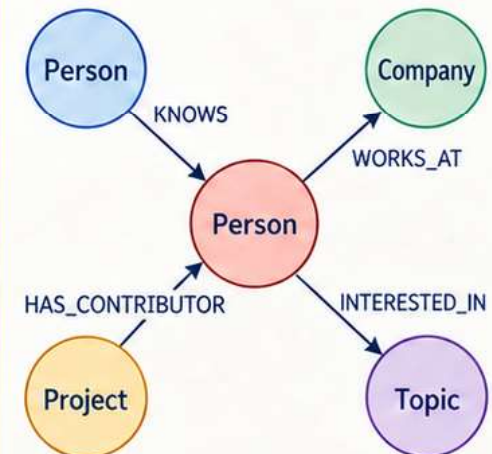
Stores wide rows grouped into column families.

Row Key	profile:name	profile:age	activity:last_login
u1	Alice	29	2025-04-12
u2	Bob	34	2025-04-10



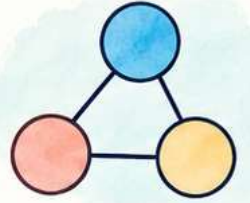
Graph Database

Stores connected entities and their relationships.

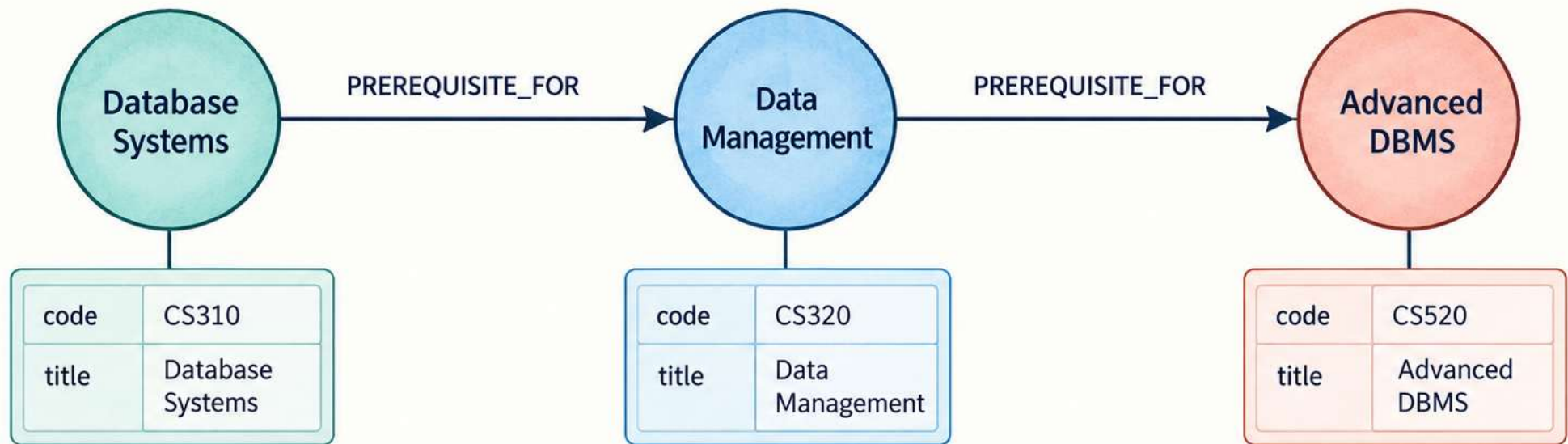


Revision note: Choose by relationships, queries, consistency needs, and operational constraints.

Property Graphs



A property graph represents entities as nodes and connections as relationships.



Nodes represent entities; relationships have types and directions; properties store details.

Graph Queries: Revision

Graph queries match patterns in connected data.

Cypher query (one-hop prerequisite)

```
MATCH (a:Course)-[:PREREQUISITE_FOR]->(b:Course)
RETURN a.title, b.title;
```

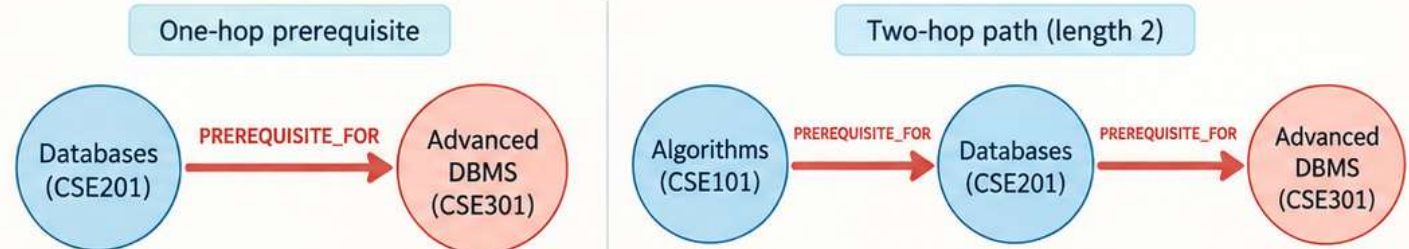
Matched pattern (one hop)



Example data (courses)

title	code
Databases	CSE201
Algorithms	CSE101
Advanced DBMS	CSE301
Machine Learning	CSE401

One-hop vs two-hop path

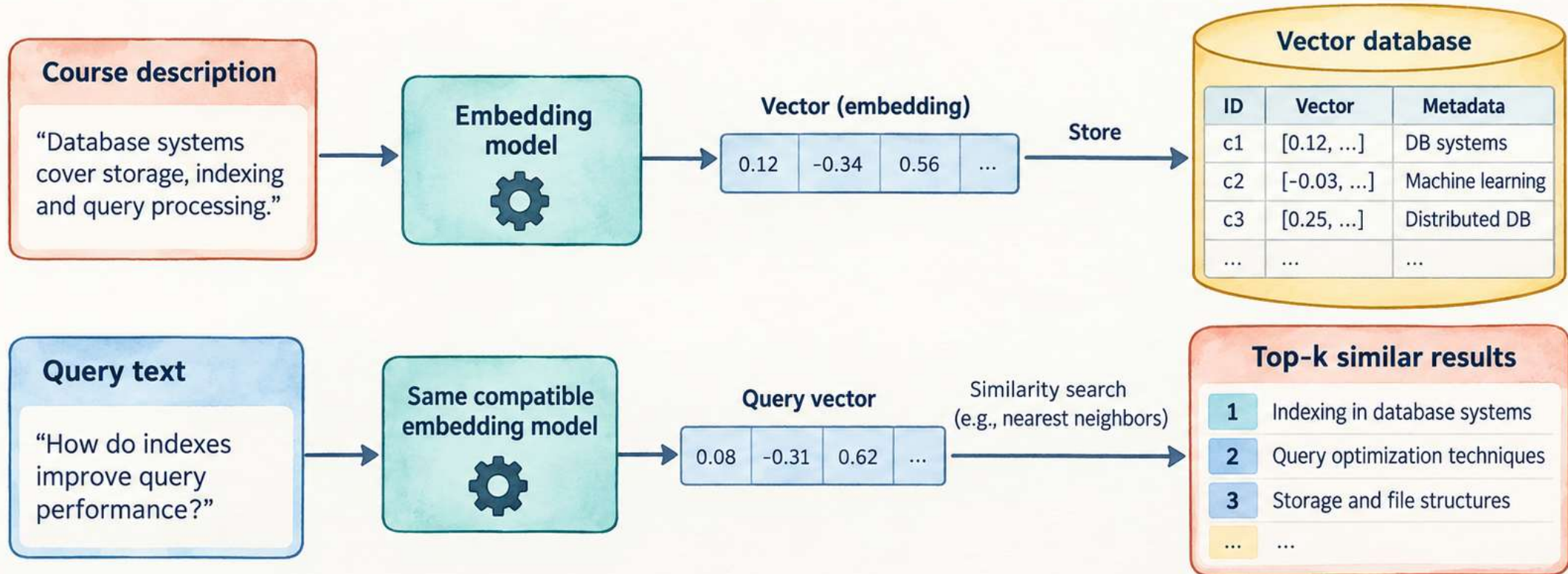


Revision

Use graphs for relationship-focused questions; keep this introduction to small patterns.

Vector Databases and Similarity Search

Vector search retrieves items whose numeric representations are close to a query representation.



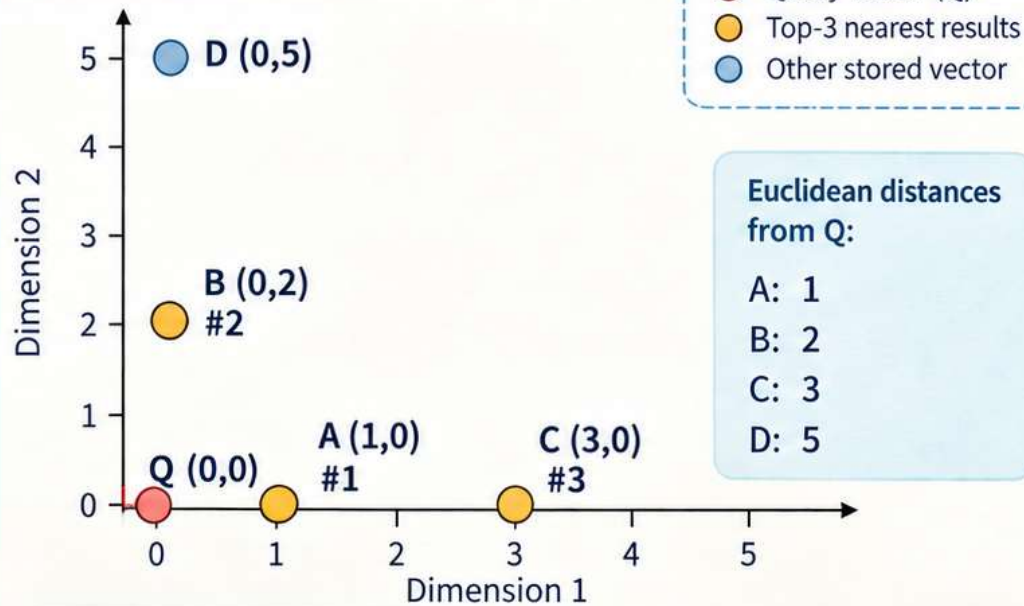
Revision: Similarity is a relevance signal, not a guarantee of factual correctness.

How Similarity Retrieval Works



Top-k search ranks stored vectors by a chosen distance or similarity measure.

Vector space (2D illustration)



Equal x/y scales; only query and four candidate vectors are shown.

Metadata filter before comparison

1. Apply metadata filter

Select only items matching relevant metadata (e.g. category, date, access).

id	category	year
A1	DBMS	2024
B7	AI	2024
C3	DBMS	2023
...	...	...

2. Compare eligible vectors

Compute distance/similarity between query and filtered vectors.

3. Matching results (up to k)

Rank by similarity (e.g. cosine similarity or Euclidean distance).

id
1. A1
2. C3



Revision: Use compatible embeddings and dimensions; filter by relevant metadata.

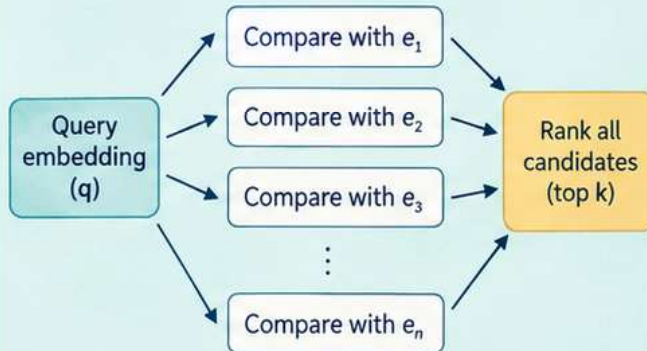


Vector Search: Revision

Start with **exact search** on a small prepared dataset.

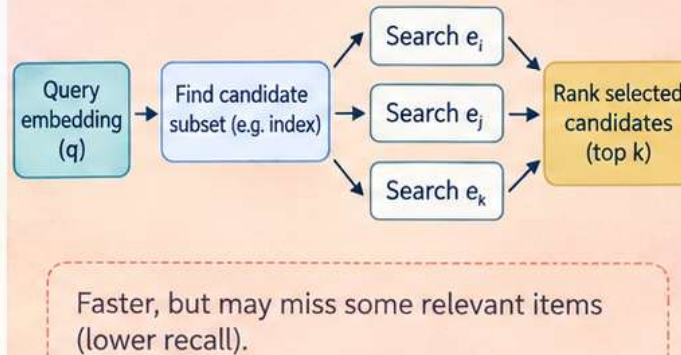
Exact Search (brute force)

Check all candidates and return the nearest.



Approximate Search

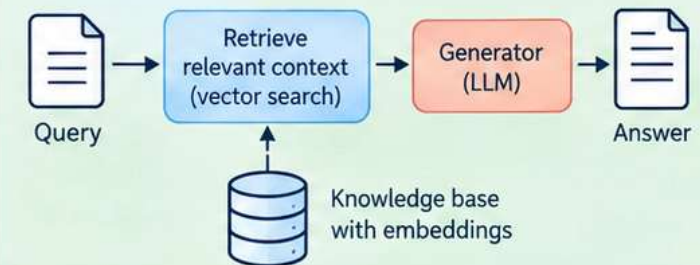
Search a selected subset of candidates, trading recall for speed.



PostgreSQL table (with embeddings)

id	description	embedding
1	DBMS overview	[0.12, 0.03, ...]
2	Query processing	[0.08, 0.41, ...]
3	Index structures	[0.25, 0.17, ...]
...	...	...

Use in RAG (optional)



Revision

Use provided embeddings; inspect relevance; defer model training and index internals.

Putting the Course Together

A sound database solution combines correct data, safe operations, and justified design choices.



Constraints

Primary keys, foreign keys, checks, referential integrity



Routines

Stored procedures, triggers, functions



Transactions

Atomicity, consistency, isolation, durability



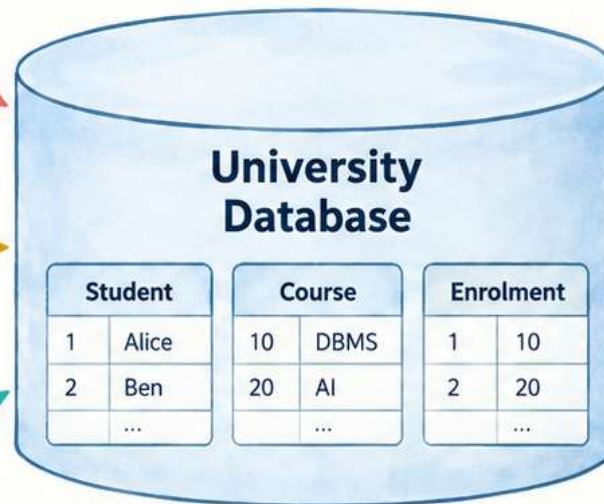
Indexes

Speed up queries, choose appropriate keys



Roles

Users, roles, privileges, least privilege



Optional extension (chosen)

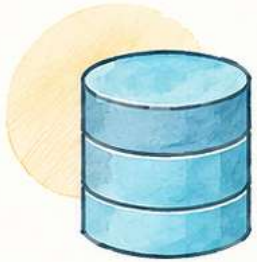


Graph Analytics

Model relationships (e.g. students collaborate) using a graph database.



Revision note: Keep the core relational solution small and explain every design choice.



Present and Defend Your Design

A project presentation explains requirements, implementation, and evidence.



1 Problem

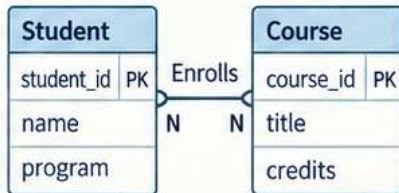
State the scenario, goals and constraints.



- What needs to be solved?
- Key requirements*
- Assumptions and limits

2 Schema

Show the main schema elements and relationships.



3 Key Operation

Demonstrate an important query or transaction.

```
SELECT s.name, c.title
FROM Student AS s
JOIN Enroll AS e
  ON s.student_id = e.student_id
JOIN Course AS c
  ON c.course_id = e.course_id
WHERE c.course_id = 'DB101';
```

4 Verification

Show evidence that it works as intended.

Query result (example)

name	title
Alice	DB101
Ben	DB101

Transaction test

- ✓ Commit persists changes
- ✗ Rollback undoes changes

5 Trade-offs

Discuss design choices and their impact.



- Performance vs. flexibility
- Normalization vs. simplicity
- Chosen optional extension
- Remaining limitations



Revision note

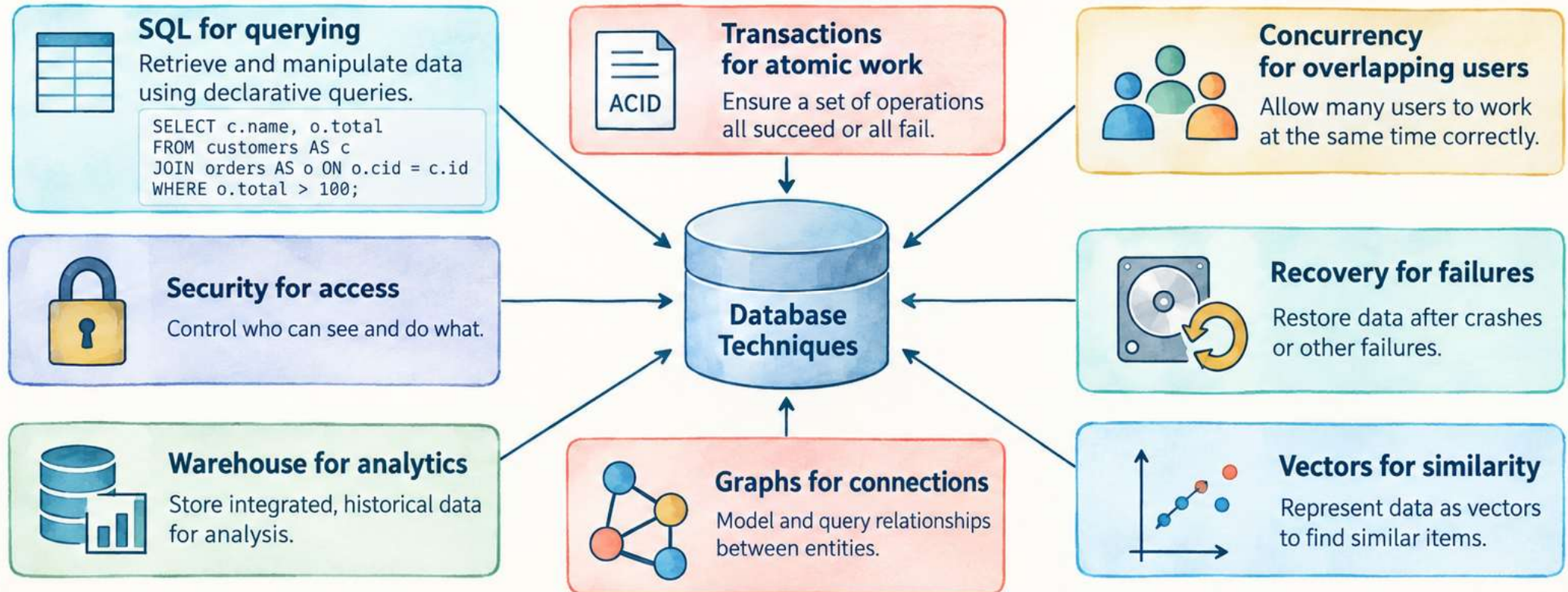
Demonstrate correctness; explain failures; justify the optional extension.



Advanced DBMS: Final Revision

45/45

Choose database techniques according to the question and the risk.

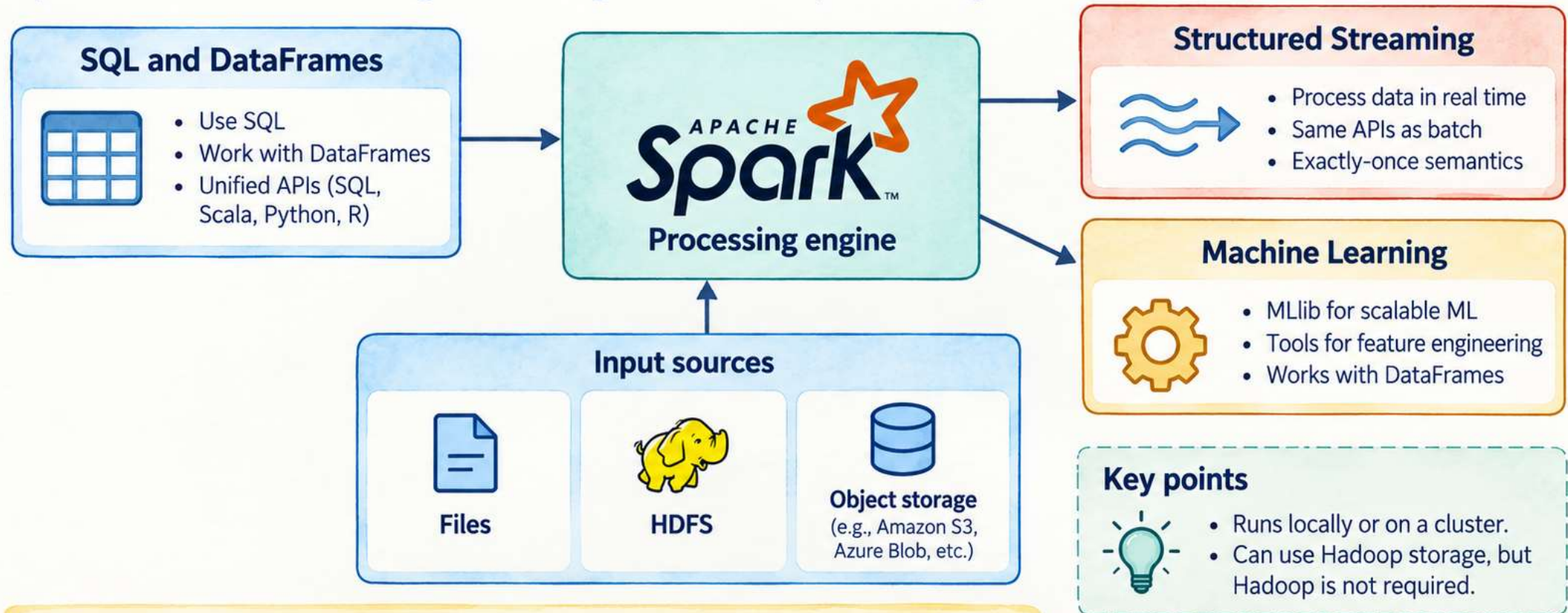


Know what each technique does, when to use it, and one important limitation.



Introducing Apache Spark

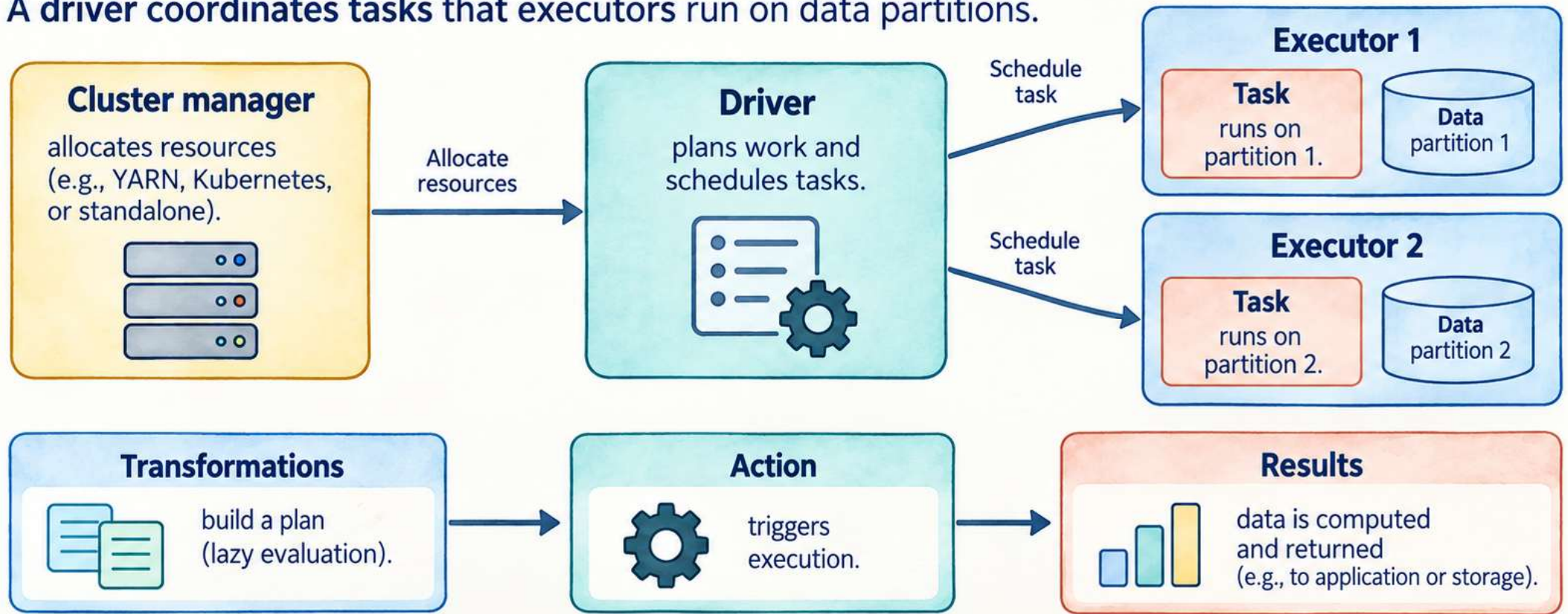
Spark is a distributed engine for large-scale data processing.



Revision: Spark processes data; it is not a storage system.

How Spark Executes Work

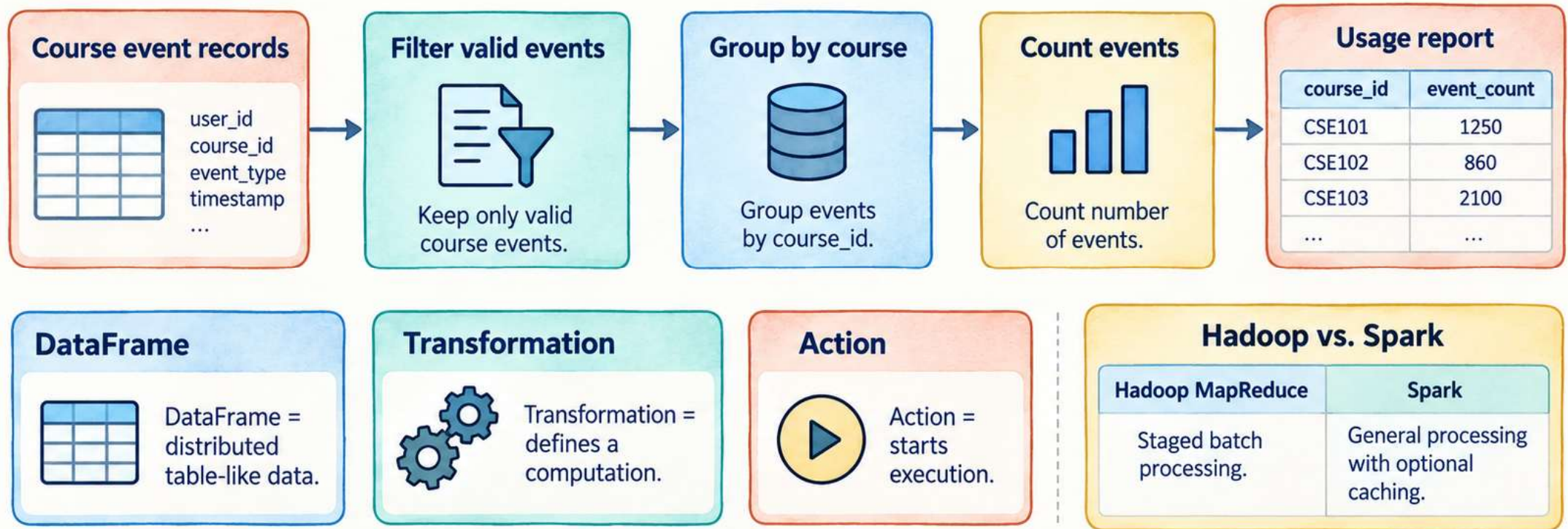
A driver coordinates tasks that executors run on data partitions.



Revision: Caching selected data can speed reuse; Spark also uses disk.

Spark SQL: Quick Revision

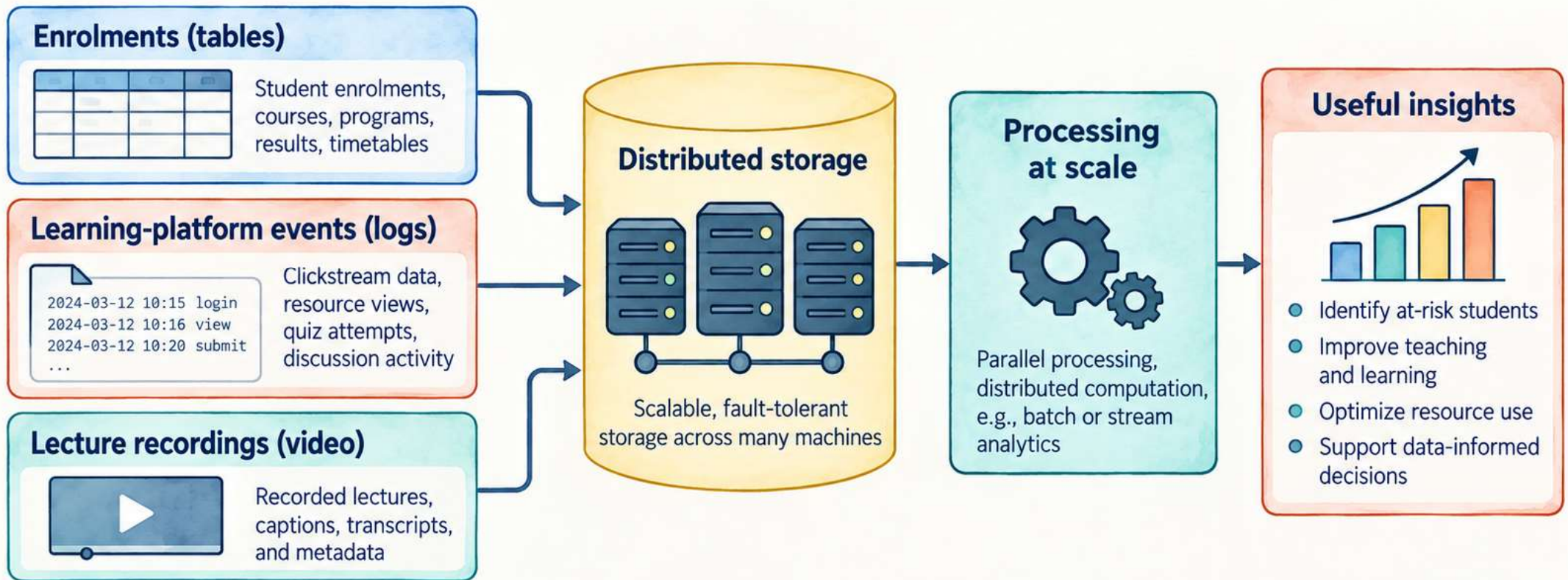
DataFrames let us transform structured data using familiar table operations.



Why can caching help when the same data is reused?

Understanding Big Data

Big data involves datasets whose scale, speed or diversity challenge conventional processing.



Revision: Big data is a workload challenge, not a specific product.

The Five Vs of Big Data

A common framework describes big data using five characteristics.

Volume

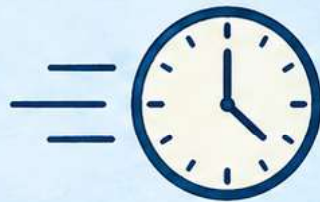
amount of data



University stores years of student records, library logs and learning resources.

Velocity

speed of arrival



Sensor and campus app data (e.g., Wi-Fi access, class check-ins) arrive in real time.

Variety

different formats



Includes text, images, videos, sensor data and social media posts from campus communities.

Veracity

trust and quality



Data may be noisy, incomplete or from unverified sources, so it needs cleaning and validation.

Value

useful outcomes



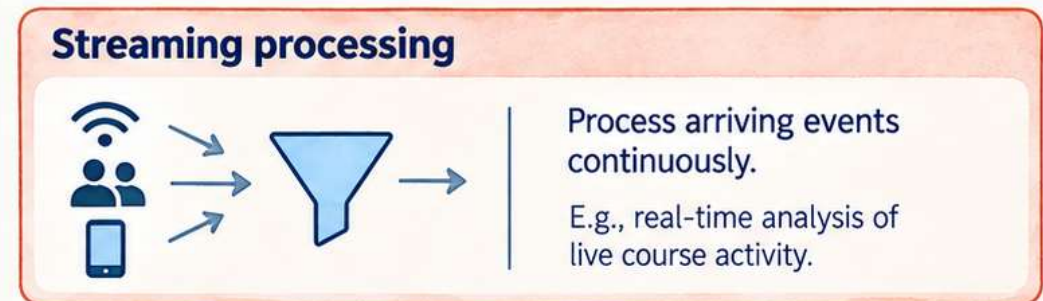
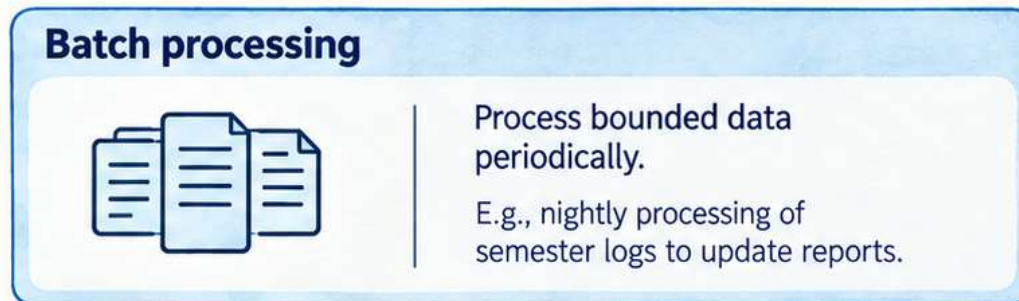
Helps improve teaching, personalise learning, optimise campus services and support better decisions.



Revision: More data is useful only when it answers a meaningful question.

Big Data: From Sources to Insights

A useful pipeline turns raw data into trustworthy answers.



Revision:

1

Start with a question.

2

Check data quality.

3

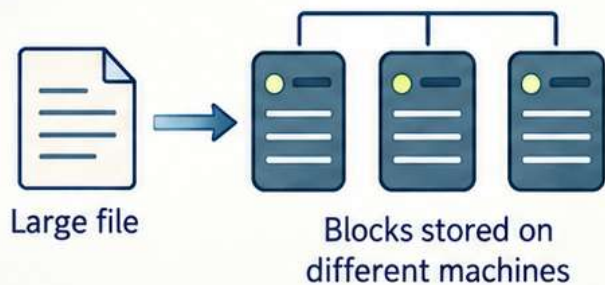
Scale only when needed.

Introducing Apache Hadoop

Hadoop provides tools for distributed storage and processing across computers.

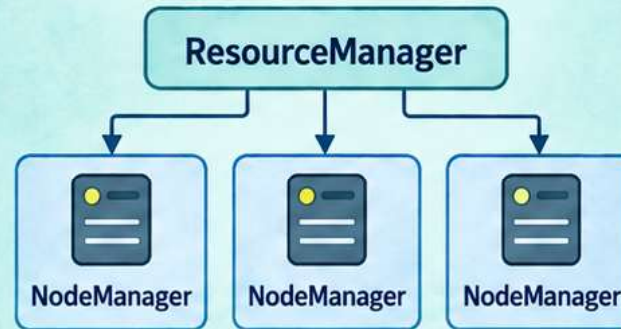
HDFS

Stores files across machines.



YARN

Manages cluster resources.



MapReduce

Processes data in parallel batches.



Hadoop Common

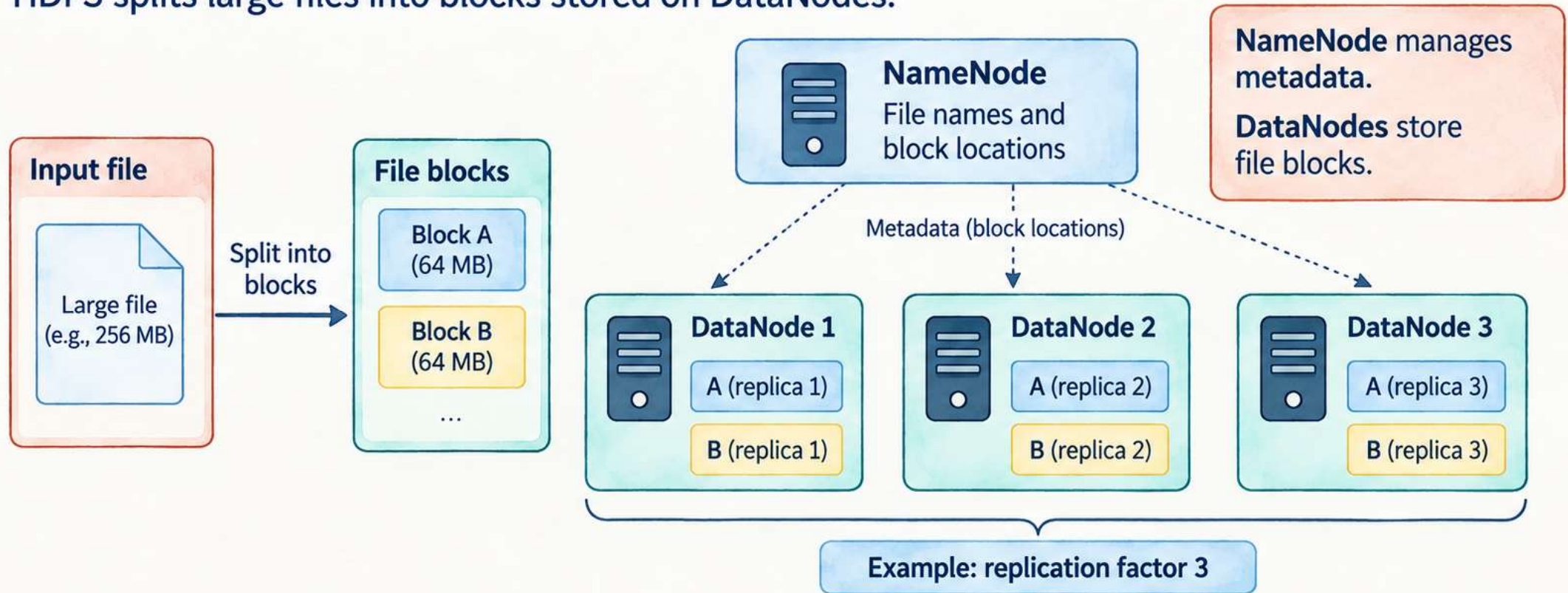
Shared utilities (e.g., configuration, libraries, I/O, and networking).



Revision: Storage, resource management and processing have different jobs.

How HDFS Stores Files

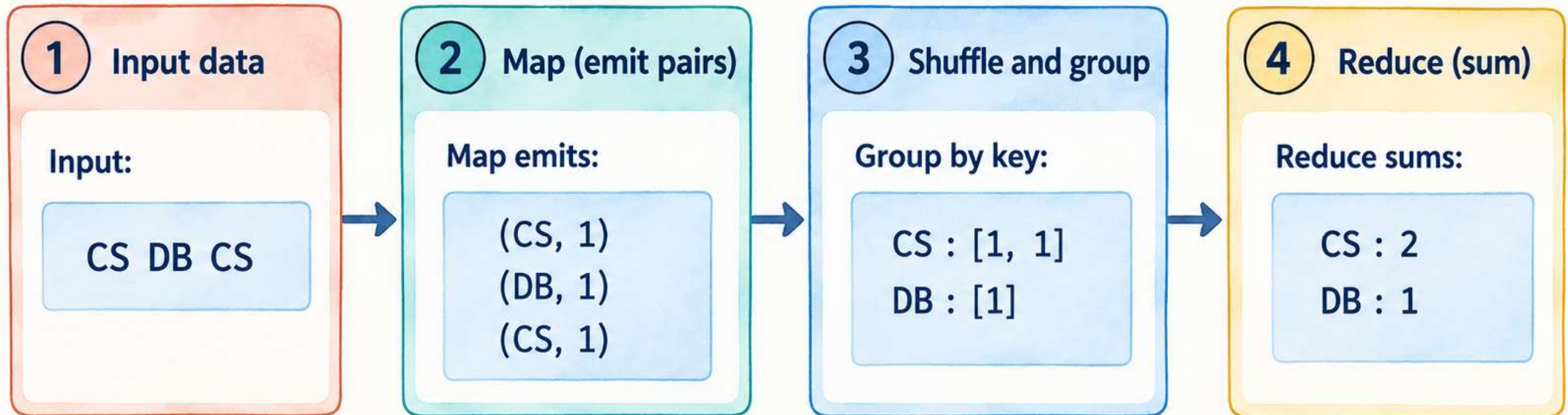
HDFS splits large files into blocks stored on DataNodes.



Revision: Replicas help tolerate a DataNode failure; replication is not a backup.

MapReduce: Split, Group, Summarize

MapReduce performs batch computations using map and reduce tasks.



Revision: HDFS stores data; YARN allocates resources; MapReduce computes results.